

國立陽明交通大學
電機資訊國際學位學程
碩士論文

EECS International Graduate Program
National Yang Ming Chiao Tung University
Master Thesis

串接 Transformer 偵測與強化學習應對的自動化威脅緩解
Streamlining Transformer-based Detection and RL-based
Response for Automated Threat Mitigation

研 究 生：翁明光（Adrian Santoso）

指導教授：林盈達（Ying-Dar Lin）

中華民國一一五年六月

June 2026

串接 Transformer 偵測與強化學習應對的自動化威脅緩解
Streamlining Transformer-based Detection and RL-based
Response for Automated Threat Mitigation

研 究 生：翁明光
指導教授：林盈達 博士

Student: Adrian Santoso
Advisor: Dr. Ying-Dar Lin

國立陽明交通大學
電機資訊國際學位學程
碩士論文

A Thesis

Submitted to EECS International Graduate Program
College of Electrical Engineering and Computer Science
National Yang Ming Chiao Tung University
in Partial Fulfilment of the Requirements
for the Degree of
Master in
Electrical Engineering and Computer Science

June 2026
Taiwan, Republic of China

中華民國 一一五年六月

串接 Transformer 偵測與強化學習應對的自動化威脅緩解

學生：翁明光

指導教授：林盈達 博士

國立陽明交通大學 電機資訊國際學位學程

摘 要

現代企業環境中的網路入侵通常會經歷多個攻擊階段，從初始入侵開始，逐步進入執行、持續化、橫向移動，最後可能造成系統衝擊。雖然資安營運中心會使用防火牆、入侵偵測系統、網路偵測與回應，以及端點偵測與回應等工具，這些工具仍會產生大量且分散的告警，使分析人員難以手動分析。現有方法多半著重於偵測或回應其中一個面向，導致攻擊者行為識別與即時緩解措施之間仍存在落差。

為了解決此限制，本論文提出 TRACE，一個雙模型自動化威脅緩解引擎，用以串流化 Transformer 式攻擊技術偵測與強化學習式回應流程。TRACE 首先關聯多來源告警，以偵測目前對應的 MITRE ATT&CK 攻擊技術，接著使用偵測到的攻擊技術、攻擊階段、系統殘留物件，以及系統狀態，來引導強化學習式回應動作選擇。此回應設計鼓勵代理人在攻擊早期採取較輕量的回應動作，並在攻擊後期允許採取較強的隔離與阻斷措施。

實驗結果顯示，Transformer 式偵測模型達到 0.856 的 F1-score，且推論延遲僅為 2.78 ms/sample；強化學習回應代理人則達到 97.5% 的系統恢復成功率。在端到端架構比較中，TRACE 的 Transformer 偵測與強化學習回應管線總延遲僅為 13.971 ms，相較之下，以大型語言模型為基礎的偵測與回應管線總延遲為 1284.29 ms。這些結果顯示，TRACE 能為即時資安營運中心環境提供準確、具適應性且低延遲的自動化威脅緩解能力。

關鍵詞：資安營運中心、MITRE ATT&CK、Transformer、強化學習、偵測與回應。

Streamlining Transformer-based Detection and RL-based Response for Automated Threat Mitigation

Student: Adrian Santoso

Advisor: Dr. Ying-Dar Lin

EECS International Graduate Program
National Yang Ming Chiao Tung University

Abstract

Cyber intrusions in modern enterprise environments often progress through multiple attack stages, from initial access to execution, persistence, lateral movement, and impact. Although Security Operations Centers use tools such as firewalls, IDS, NDR, and EDR, these tools generate fragmented alerts that are difficult to analyze manually. Existing approaches often focus on either detection or response, leaving a gap between identifying attacker behavior and taking timely mitigation actions.

To address this limitation, this thesis proposes TRACE, a dual-model automated threat mitigation engine that streamlines Transformer-based technique detection and reinforcement learning-based response. TRACE correlates multi-source alerts to detect the current MITRE ATT&CK technique, then uses the detected technique, attack stage, system artifacts, and system status to guide RL-based response action selection. The response design encourages lightweight actions in early attack stages while allowing stronger containment actions in later stages.

Experimental results show that Transformer-based detection achieves an F1-score of 0.856 with 2.78 ms/sample latency, while the RL response agent reaches a 97.5% system recovery success rate. In the end-to-end comparison, the TRACE Transformer-detection and RL-response pipeline achieves only 13.971 ms total latency, compared with 1284.29 ms for the LLM-based detection-and-response pipeline. These results demonstrate that TRACE provides accurate, adaptive, and low-latency automated threat mitigation for real-time SOC environments.

Keywords: Security Operations Center, MITRE ATT&CK, Transformer, Reinforcement Learning, Detection & Response.

Acknowledgement

回顧這段研究旅程，我心中充滿感激。這篇論文的完成並不輕鬆，期間有疑惑、壓力、不眠的夜晚，也曾有想要放棄的時刻。然而，因為身邊許多人的鼓勵、陪伴與支持，我才能堅持走到最後。

首先，我要向我的指導教授林盈達老師表達最深的感謝。您對「為什麼」的不斷追問，不僅塑造了我研究的邏輯，也影響了我面對問題時的思考方式。您教會我以更精確、更有耐心的態度探索問題，同時保有對研究的好奇心。這些學習不僅幫助我完成本篇論文，也將成為我未來道路上的重要養分。

我也誠摯感謝賴源正老師與黃仁竑老師。您們在定期討論中給予的指導與建議，幫助我修正與完善研究內容，也讓我更加理解清晰表達與嚴謹思考的重要性。您們的寶貴意見，使這篇論文更加完整，也讓我在研究過程中獲益良多。

感謝 701 MIRC 實驗室的所有夥伴們。您們不僅是一起工作的同事，更是陪伴我走過這段研究旅程的重要存在。特別感謝學長們的指導、經驗分享與鼓勵，讓我在遇到困難時能夠重新整理方向並繼續前進。也謝謝室友、同學與學弟妹們的歡笑、討論與陪伴。這些共同努力的時光，將會是我求學過程中珍貴的回憶。

我也要感謝我的家人與分布在世界各地的摯友們。即使相隔遙遠，您們始終如一的關心、支持與鼓勵，依然是我最堅實的後盾，也在我疲憊與迷惘時給予我力量與溫暖。這份親情、友情與陪伴，讓我知道無論身在何處，我都不是獨自面對這段旅程。

最後，我要感謝自己。這是一段充滿挑戰的旅程，無論在學術、情緒或生活上，都有許多需要獨自面對與克服的時刻。我曾在實驗室與房間裡度過無數個夜晚，靠著咖啡與堅持撐過一次又一次的困難。這篇論文不僅是研究成果的體現，更是支持、陪伴、成長與韌性的結晶。謹以此篇致謝，獻給所有曾經幫助、鼓勵與陪伴我的人。

Table of Contents

摘要	i
Abstract	ii
Acknowledgement	iii
Table of Contents	iv
List of Figures	vi
List of Tables	vii
1 Introduction	1
2 Background and Related Work	4
2.1 Cybersecurity Context and Attack Lifecycle Modeling	4
2.1.1 Security Operations Center	4
2.1.2 MITRE ATT&CK Framework	5
2.1.3 CTI-to-Lifecycle Extraction with MITREtrieval	6
2.2 AI-Based Threat Detection and Automated Response	7
2.2.1 Transformer-Based Technique Detection	7
2.2.2 Reinforcement Learning for Automated Response	8
2.2.3 AI Agent and Automation Architectures	8
2.3 Related Work	9
3 Problem Formulation	12
3.1 Notations	12
3.2 Problem Statement	14
3.2.1 Technique Detection	15
3.2.2 Response Decision	16
4 TRACE: Transformer-RL Automated Cybersecurity Engine	17
4.1 Solution Overview	17
4.2 Transformer-Based Technique Detection	18
4.3 RL-Based Response Decision	20

4.3.1	Response Action Space	20
4.3.2	Reward Function	21
4.3.3	Stage-Aware Response Design	23
4.3.4	Stage-Aware DDQN Response Algorithm	24
4.4	Example Run	25
5	Implementation	27
5.1	Open-Source Tools and Libraries	27
5.2	Testbed Setup	28
5.3	Implementation Details	29
5.3.1	Sighting Feature Construction	30
5.3.2	Response-State Implementation	31
6	Evaluation and Results	33
6.1	Baseline	33
6.1.1	Detection Model Configuration	33
6.1.2	Response Model Configuration	34
6.1.3	LLM Detection and Response Baseline	35
6.2	Data Profiling	37
6.3	Technique Detection Performance	38
6.4	RL Response Decision	40
6.4.1	Recovery Learning Curve	40
6.4.2	Stage-Aware Action Selection	41
6.4.3	Lifecycle Stopping Behavior	42
6.5	Response Model Comparison: RL vs. LLM	43
6.5.1	Recovery Success Rate	43
6.5.2	Stage-Aware Action Behavior	44
6.5.3	Lifecycle Stopping Comparison	46
6.6	End-to-End Architecture Comparison	47
7	Conclusion and Future Work	49
	References	52

List of Figures

2.1	Security Operations Center workflow for centralized cyber defense.	5
2.2	MITRE ATT&CK tactic, technique, and attack lifecycle representation.	6
2.3	CTI-to-technique extraction process using MITREtrieval.	7
2.4	Reinforcement learning feedback loop.	8
2.5	AI agent-based cybersecurity automation architecture.	9
3.1	Overall problem formulation of the TRACE detection–response pipeline. . . .	14
4.1	Overview of the TRACE detection–response framework.	18
4.2	Transformer-based technique detection process.	19
4.3	Example run of TRACE from attack lifecycle detection to response action selection.	26
5.1	Implemented system architecture of the TRACE detection-and-response pipeline.	30
5.2	System status determination from artifact states.	32
6.1	LLM-based detection and response pipeline.	36
6.2	Example of LLM response prompt design.	37
6.3	Lifecycle length distribution of CTI-derived attack scenarios.	38
6.4	LLM and Transformer performance in MITRE ATT&CK technique detection. .	39
6.5	RL learning curve of system recovery success rate per episode.	40
6.6	RL action-impact distribution across attack stages.	42
6.7	Lifecycle stopping distribution by tactic under different RL stage designs. . . .	43
6.8	System recovery success rate of RL and LLM response models.	44
6.9	Action-impact distribution of RL and LLM response models across attack stages.	45
6.10	Lifecycle stopping distribution by tactic for RL and LLM response models. . .	46
6.11	End-to-end comparison of Transformer+RL and LLM-based detection–response pipelines.	48

List of Tables

2.1	Comparison of Related Work on Threat Detection and Response	10
3.1	Notations Used in the Problem Formulation	13
4.1	Response Action Space Grouped by Operational Impact Level	21
4.2	Combined Reward Design for the TRACE Response Agent	23
4.3	Stage Definition Used in the Stage-Aware Response Design	24
5.1	Open-Source Tools and Libraries Used in the Implementation	28
5.2	Selected Sighting Features from Security Tools	30
5.3	RL System Artifact State Representation	31
6.1	Transformer and LLM Detection Model Configuration	34
6.2	RL Response Model Configuration	35
6.3	LLM Response Model Configuration	36
6.4	Collected Sighting Statistics	38

Chapter 1

Introduction

Cyber intrusions in real-world environments often unfold as multi-stage attack lifecycles, starting from initial access and progressing toward final impact [1]. To defend against these threats, enterprises deploy multiple security tools, including firewalls (FW), Intrusion Detection Systems (IDS), Network Detection and Response (NDR), and Endpoint Detection and Response (EDR). However, these tools generate large volumes of alerts across network and host environments [2]. As a result, Security Operations Centers (SOCs) face alert fatigue [3], making it difficult for analysts to manually inspect every alert, connect fragmented evidence across sources, and understand how observed behaviors fit into the broader attack progression.

The key challenge is therefore to transform fragmented multi-source alerts into a higher-level representation that can support response decisions. MITRE ATT&CK technique detection provides this representation by identifying the attacker’s current behavior from correlated FW, IDS, NDR, and EDR alerts. This technique-level view helps the system understand where the attacker is in the attack lifecycle and provides more useful context for mitigation. AI-based detection can support this process by learning patterns from heterogeneous alerts and mapping them to attack techniques with lower manual effort [4].

Detection alone, however, is not enough to stop an attack. If the system still relies on manual analysis or fixed predefined actions after detection, response can become another bottleneck. Fixed responses may under-respond and allow the attacker to continue, or over-respond and disrupt normal operations. Effective mitigation therefore requires TRACE, a streamlined AI-based design that links low-latency technique detection with adaptive response decision-making. In this thesis, the response design focuses on stage-aware mitigation. The RL agent uses attack progression to adjust response strength: early-stage attacks are encouraged to use lighter actions, whereas later-stage attacks allow stronger containment actions such as endpoint isolation

or workstation shutdown to prevent further damage. To highlight the effect of stage information, this thesis compares the proposed stage-aware design with a risk-aware response baseline, following the common design used in prior automated response studies. In this risk-aware setting, the response policy considers the impact of each action on the operational environment; therefore, it tends to favor low-impact actions regardless of whether the attack is in the early, middle, or late stage.

Existing studies still do not fully address this end-to-end requirement. Many detection methods focus on attack classification using a single alert source such as IDS [5], which limits their ability to correlate heterogeneous evidence. Some recent studies support alert correlation [4], but they mainly focus on detection and do not close the loop with automated mitigation. On the response side, RL-based methods have been explored for stopping attacks and restoring compromised systems [6, 7], but they often do not explicitly adjust response strength according to attack stage. LLM-based approaches are also promising for flexible response generation [8], but their latency and operational cost can limit their suitability for real-time SOC environments. Therefore, a research gap remains in building TRACE, which streamlines multi-source technique detection, low-latency automated response, and stage-aware mitigation.

To address this gap, we propose TRACE, a dual-model system architecture that streamlines Transformer-based detection and RL-based response. TRACE ingests heterogeneous alerts from FW, IDS, NDR, and EDR systems, correlates evidence across sensors, and uses a Transformer model to identify the current MITRE ATT&CK technique in near real time. The detected technique is then used to derive the current attack stage and response context. Based on the stage, system artifacts, and system status, the RL agent selects a mitigation action whose strength matches the attack progression and current host condition. This design enables the system to move beyond attack recognition and perform autonomous mitigation aimed at restoring the host to a normal state and stopping the attack lifecycle earlier. By streamlining detection and response, TRACE shifts SOC operation from passive monitoring toward proactive and automated threat defense.

This thesis evaluates TRACE from four aspects. First, technique detection is evaluated by comparing Transformer-based models and LLMs using F1-score, accuracy, recall, and inference time. Second, RL-based response is evaluated in terms of recovery success, suitable action selection across attack stages, and lifecycle stopping. Third, RL-based response is compared with LLM-based response to analyze which model provides more consistent recovery, more suitable action selection across attack stages, and better lifecycle stopping. Fourth, the full Transformer-plus-RL architecture is compared with an LLM-based detection-and-response pipeline using detection latency, response latency, and end-to-end latency. These evaluations measure not only detection accuracy, but also response effectiveness, lifecycle reduction, stage-aware behavior, and real-time feasibility for SOC environments.

The contributions of this thesis are summarized as follows. First, we propose TRACE, a streamlined detection-and-response engine that connects multi-source alert correlation, MITRE ATT&CK technique detection, and automated mitigation into a single workflow. Second, we design a low-latency dual-model architecture for SOC environments by streamlining Transformer-based technique detection with RL-based response decision-making. The Transformer model provides fast technique identification under high alert volume, while the RL agent selects mitigation actions based on attack stage, system artifacts, and system status. Third, we evaluate TRACE using CTI-derived attack lifecycles and operational metrics covering detection accuracy, response effectiveness, lifecycle stopping behavior, suitable action selection across attack stages, and end-to-end latency. The results demonstrate that TRACE supports accurate detection, adaptive stage-aware response, and low-latency automated mitigation.

The remainder of this thesis is organized as follows. Chapter 2 discusses background and related work. Chapters 3 and 4 present the problem statement and TRACE solution. Chapter 5 describes the system implementation, and Chapter 6 presents the evaluation and experimental results. Finally, Chapter 7 concludes the thesis and discusses future work.

Chapter 2

Background and Related Work

This chapter introduces the cybersecurity and AI concepts used in this thesis. Section 2.1 explains SOC operation, MITRE ATT&CK, and CTI-to-lifecycle extraction with MITREtrieval. These concepts show how attacker behavior can be represented as structured technique sequences. Section 2.2 reviews Transformer-based detection, RL-based response, and AI automation architectures. Section 2.3 compares related studies and identifies the research gap.

2.1 Cybersecurity Context and Attack Lifecycle Modeling

2.1.1 Security Operations Center

A Security Operations Center (SOC) monitors, detects, analyzes, and responds to cyber threats across an organization [9]. It is the main operational environment for cyber defense. Its goal is to maintain visibility, detect suspicious activity, investigate incidents, and coordinate timely response.

SOCs collect telemetry from tools deployed across the environment. Common sources include Intrusion Detection Systems (IDS), firewalls (FW), Network Detection and Response (NDR), and Endpoint Detection and Response (EDR). These tools provide different views of attacker activity, from network traffic to host-level events. Combining them gives analysts broader visibility across network and endpoint systems.

However, modern SOC face high alert volume, heterogeneous data, and slow manual investigation. Analysts must connect related evidence across tools before choosing a response. This motivates automated methods that support both detection and response in SOC environments. Figure 2.1 summarizes this SOC workflow.

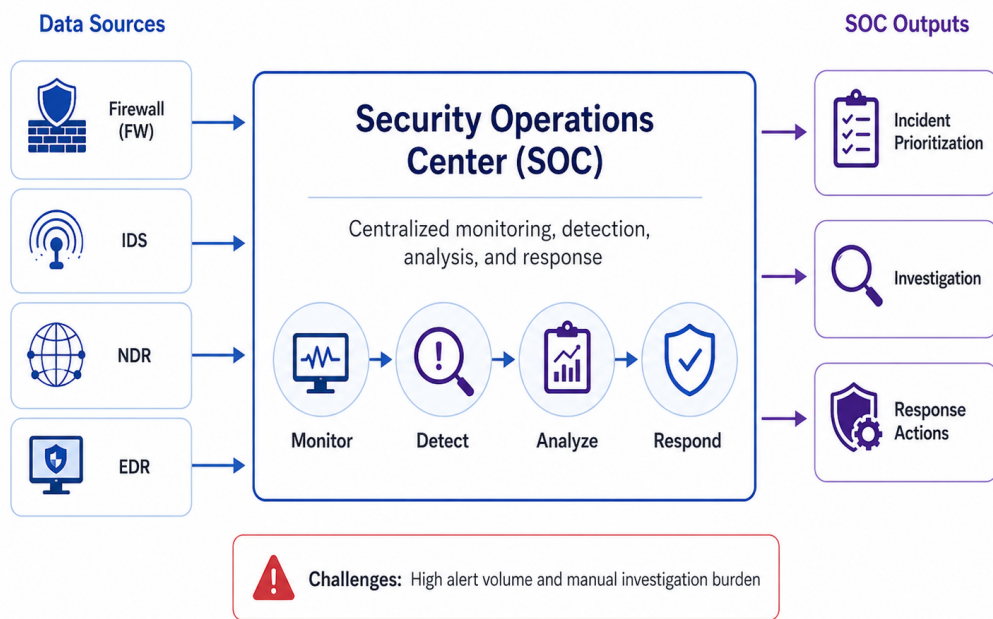


Figure 2.1: Security Operations Center workflow for centralized cyber defense.

2.1.2 MITRE ATT&CK Framework

The MITRE ATT&CK framework is a structured knowledge base of real-world adversary behavior [10]. It provides a common language for threat analysis, detection engineering, and incident response. In ATT&CK, a tactic describes the attacker's objective, while a technique describes how the objective is achieved. Procedures provide concrete examples of technique implementation.

ATT&CK is useful because technique sequences can represent an attack lifecycle. Instead of treating alerts as isolated events, analysts can connect them into a broader attack progression. This is important for multi-stage intrusions that move from initial access to execution, persistence, lateral movement, exfiltration, or impact.

In this thesis, ATT&CK provides two main benefits. First, it gives a standard representation for mapping alerts to attacker behavior. Second, it supports lifecycle-based reasoning, which helps the system interpret a detected technique within the attack progression. Figure 2.2 shows the relationship among tactics, techniques, and lifecycles.

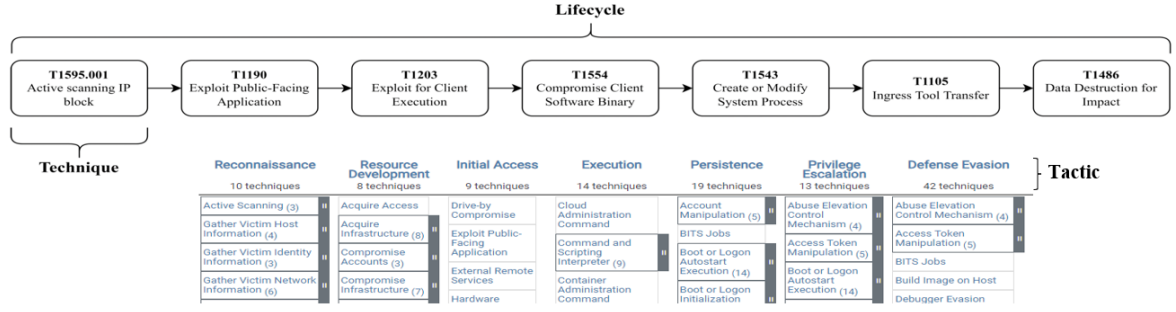


Figure 2.2: MITRE ATT&CK tactic, technique, and attack lifecycle representation.

2.1.3 CTI-to-Lifecycle Extraction with MITREtrieval

Cyber Threat Intelligence (CTI) is the collection and use of threat-related information to improve cyber defense [11]. CTI helps defenders understand attacker behavior, improve detection, and plan response. It may include Indicators of Compromise (IoCs), such as IP addresses, domains, URLs, and file hashes, as well as Tactics, Techniques, and Procedures (TTPs). CTI connects raw security events with attacker intent. Local alerts often show isolated evidence, while CTI provides higher-level knowledge about campaigns and adversary behavior. In this thesis, CTI is also used to construct attack lifecycle data.

Although CTI is valuable, much of it appears as unstructured text in threat reports and campaign analyses. This makes it difficult to use directly in automated systems. MITREtrieval addresses this by extracting MITRE ATT&CK techniques from CTI reports and converting them into structured technique sequences [12]. MITREtrieval identifies attacker techniques at the sentence level and organizes them into lifecycles. This converts narrative threat reports into machine-readable representations of adversary behavior. These structured lifecycles can then support security analytics, model training, and attack simulation.

In this work, MITREtrieval bridges external CTI knowledge and the lifecycle data used by TRACE. The extracted lifecycles are used to generate realistic attack sequences for the testbed. They also support detection model fine-tuning and evaluation. Figure 2.3 shows this conversion process.

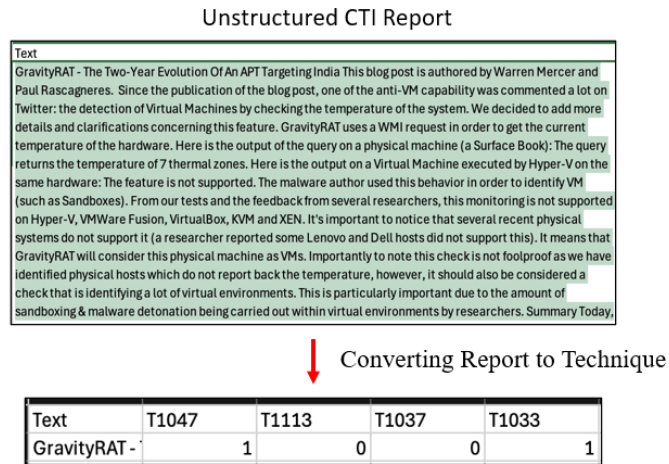


Figure 2.3: CTI-to-technique extraction process using MITREtrieval.

2.2 AI-Based Threat Detection and Automated Response

2.2.1 Transformer-Based Technique Detection

Transformer models are effective for text understanding because they learn contextual relationships in sequential data [13]. This is useful in cybersecurity because alerts, CTI reports, and ATT&CK descriptions are often textual or semi-structured. Transformer models can learn these patterns and map them to attacker techniques.

Compared with large LLM-based approaches, Transformer-based models can be more lightweight and faster during inference [14]. This is important for SOCs that must process many alerts quickly. Through fine-tuning, Transformer models can also learn cybersecurity terminology and attack-related patterns [15].

In this thesis, alerts from firewall, IDS, NDR, and EDR systems are used to identify the current MITRE ATT&CK technique. The detection module converts fragmented multi-source alerts into a technique-level representation. This detected technique is then used to build the response context.

2.2.2 Reinforcement Learning for Automated Response

Reinforcement Learning (RL) is a learning method in which an agent selects actions and receives rewards from an environment [16]. In this thesis, RL is used after technique detection to select mitigation actions automatically. This fits cyber response because each action can change the system state and affect the next attack step.

The basic RL process includes an agent, an environment, actions, and rewards. The agent observes the current state, selects an action, receives feedback, and observes the updated state. Here, the goal is to recover compromised hosts while avoiding unnecessary disruption.

RL is useful because it can learn adaptive response policies instead of fixed rules. The agent can learn which actions are suitable for different attack stages and system conditions. This supports stage-aware response, where lighter actions are preferred early and stronger containment becomes acceptable later. Figure 2.4 shows the RL feedback loop.

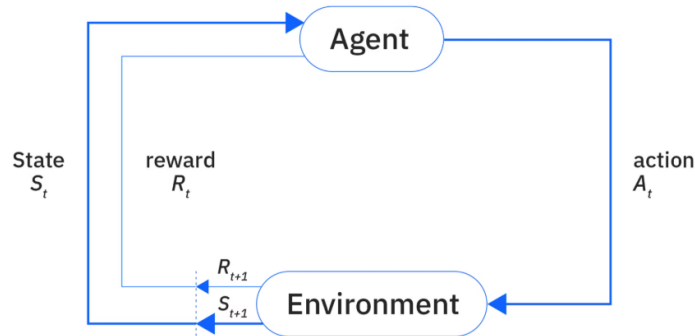


Figure 2.4: Reinforcement learning feedback loop.

2.2.3 AI Agent and Automation Architectures

AI agents are software entities that can reason, use tools, and perform actions in an environment [17]. Unlike conventional AI models that mainly output predictions, AI agents can coordinate multi-step operations. These operations may include calling APIs, executing scripts, retrieving information, or interacting with external services.

In this thesis, AI agent and automation architectures show how detection and response can be streamlined. TRACE does not stop at attack identification. After a technique is detected,

the system selects and executes suitable mitigation actions. This supports a shift from passive monitoring to streamlined cyber defense.

Detection outputs, knowledge bases, security tools, external services, and command execution can be connected to produce mitigation actions. This idea supports TRACE, where detection outputs guide automated response decisions. Figure 2.5 illustrates this automation architecture.



Figure 2.5: AI agent-based cybersecurity automation architecture.

2.3 Related Work

Related studies are compared based on detection, response, and streamlined detection-and-response capabilities. For detection, the comparison focuses on attack classification and alert correlation. For response, it considers automatic response, risk-aware response, and stage-aware response. Table 2.1 summarizes their focus, model type, dataset, and supported capabilities.

Table 2.1: Comparison of Related Work on Threat Detection and Response

Paper	Study Focus	Detection		Response			Model	Dataset
		Attack Type Classification	Alert Correlation	Automatic	Risk-aware	Stage-aware		
[4]	Detection	✓	✓	–	–	–	LLM	IDS, Firewall, EDR, NDR
[5]		✓	–	–	–	–	SL	IDS
[18]		–	–	–	–	–	DL	IDS
[7]	Response	–	–	✓	–	–	RL	–
[6]		–	–	✓	–	–	RL	Network
[19]		–	–	✓	✓	–	Rule-based	IDS
[20]		–	–	✓	✓	–	RL	Endpoint
[8]	Detection +	✓	✓	–	✓	–	LLM	IDS, EDR
Ours	Response	✓	✓	✓	✓	✓	Transformer + RL	IDS, Firewall, EDR, NDR

From the detection perspective, some studies remain coarse-grained and do not identify specific ATT&CK techniques. Lee et al. [18] convert security logs into event profiles and use neural models to detect threats, but the output mainly shows whether an event is malicious. Muhammad et al. [5] combine SIEM and IDS data for live DoS detection, but the task is narrower than multi-technique ATT&CK recognition. Lin et al. [4] use an LLM with LoRA adaptation to map heterogeneous telemetry to ATT&CK-aligned lifecycle outputs. However, LLM-based detection may introduce higher latency in high-volume SOC environments.

From the response perspective, several studies treat cyber response as an automated decision problem. Hughes et al. [7] use RL to stop attacks and restore compromised systems. Malialis and Kudenko [6] apply RL to adaptive DDoS mitigation. Hughes et al. [19] select countermeasures using IDS alerts and structured knowledge, while Iturbe et al. [20] apply RL-based response in endpoint environments. Ismail et al. [8] propose an LLM-based SOAR framework, but the response still requires user validation before execution.

Risk-aware response is another important direction. Hughes et al. [19], Iturbe et al. [20], and Ismail et al. [8] consider the impact of response actions on the defended environment. Cost-benefit or risk-based reasoning can reduce unnecessary disruption. However, many studies do

not explicitly adjust response strength according to attack progression.

This motivates stage-aware response. In this thesis, the response module uses attack progression to decide when stronger actions are appropriate. Early-stage attacks may use lighter actions, while late-stage attacks may require stronger containment. In contrast, risk-aware response mainly evaluates action impact and may favor low-impact actions regardless of attack stage. This distinction helps analyze how stage information affects action selection and lifecycle stopping.

Overall, prior work has improved threat detection and automated response, but gaps remain. Detection-focused studies often use limited alert sources or do not connect detection to mitigation. Response-focused studies may automate action selection or consider risk, but often do not adapt action strength across attack stages. LLM-based approaches provide flexible reasoning, but their latency, cost, and validation needs can limit real-time SOC use.

To address these gaps, this thesis proposes TRACE as a streamlined detection-and-response engine. The detection module correlates IDS, firewall, EDR, and NDR alerts to identify the current MITRE ATT&CK technique. The response module then selects mitigation actions using attack stage, system artifacts, and system status. Therefore, TRACE supports attack classification, multi-source alert correlation, automatic response, and stage-aware action selection in one SOC-oriented pipeline.

Chapter 3

Problem Formulation

This chapter formulates the problem addressed in this thesis: automated threat detection and response in Security Operations Centers (SOCs). It first defines the notations used in the TRACE detection–response pipeline. It then presents the overall problem by connecting multi-source alert analysis, MITRE ATT&CK technique detection, and reinforcement-learning-based response decision-making.

3.1 Notations

Table 3.1 summarizes the notations used to describe the TRACE detection–response problem. The notations follow the same flow as TRACE: CTI-derived lifecycles define attack progression, multi-source alerts provide observations for technique detection, and the response module uses the detected context to select mitigation actions.

The formulation begins from the CTI side. Let $L = \{l_i \mid i = 0, \dots, N_L - 1\}$ denote the finite set of attack lifecycles, where N_L is the number of lifecycles. Each lifecycle $l_i = \{T_{i,0}, \dots, T_{i,N_i-1}\}$ is an ordered sequence of MITRE ATT&CK techniques. This lifecycle view describes how an attack may move from one technique to another and supports response decisions based on the current detected technique.

To observe the attack in the SOC environment, alerts are collected from multiple security sensors. The sensor index set is written as $R = \{1 : \text{FW}, 2 : \text{IDS}, 3 : \text{NDR}, 4 : \text{EDR}\}$, representing firewall, intrusion detection system, network detection and response, and endpoint detection and response sources. A single event generated by sensor R at timestamp t is denoted as $E_{t,R}$. Since one event usually provides only partial evidence, events are grouped into the m -th time window W_m , and the multi-source alert set in that window is denoted by X_m . The detection model f_θ uses this alert set to infer the attacker’s current behavior. For step m in lifecycle l , $T_m^{(l)}$ denotes the ground-truth MITRE ATT&CK technique, while $\hat{T}_m^{(l)}$ denotes the

technique predicted by the detection model.

After technique detection, the notation moves to the response decision problem. The possible system statuses are represented by $Z = \{1 : \text{NORMAL}, 2 : \text{COMPROMISED}\}$, and the attack stages are represented by $G = \{1 : \text{EARLY}, 2 : \text{MID}, 3 : \text{LATE}\}$. The response module chooses from a predefined action space, where N_A is the number of available actions and $A = \{A_0, \dots, A_{N_A-1}\}$ denotes the full action set. At step m of lifecycle l , the compact RL state is denoted by $\text{St}_m^{(l)}$ and consists of the current attack stage $G_m^{(l)}$, system artifact set $F_m^{(l)}$, and system status $Z_m^{(l)}$. The reward after applying a response is denoted by $R_m^{(l)}$, the selected action is denoted by $a_m^{(l)} \in A$, and the learned response policy is denoted by π_ϕ . In addition, $\text{Prompt}^{(l)}$ denotes the LLM response prompt used for lifecycle l in the comparison experiment. These notations connect the detection and response tasks while preserving the design that the detected technique is used to construct the response context.

Table 3.1: Notations Used in the Problem Formulation

Category	Symbol	Description
CTI	$T_{i,j}$	The j -th technique in lifecycle l_i .
	N_i	Number of techniques in lifecycle l_i .
	$l_i = \{T_{i,0}, \dots, T_{i,N_i-1}\}$	Lifecycle l_i as an ordered sequence of techniques.
	N_L	Number of available lifecycles.
	$L = \{l_i \mid i = 0, \dots, N_L - 1\}$	Finite set of lifecycles.
Alert & Detection Module	$R = \{1 : \text{FW}, 2 : \text{IDS}, 3 : \text{NDR}, 4 : \text{EDR}\}$	Index four sensor types: FW, IDS, NDR, and EDR.
	$E_{t,R}$	Security event generated by sensor R at timestamp t .
	W_m	The m -th alert time window.
	X_m	Multi-source alert set in time window W_m .
	f_θ	Detection model parameterized by θ .
	$T_m^{(l)}$	Ground-truth technique at step m in lifecycle l .
	$\hat{T}_m^{(l)}$	Technique detected at step m for lifecycle l .
Response Module	$Z = \{1 : \text{NORMAL}, 2 : \text{COMPROMISED}\}$	Set of possible system statuses.
	$G = \{1 : \text{EARLY}, 2 : \text{MID}, 3 : \text{LATE}\}$	Set of possible attack stages.
	N_A	Number of predefined response actions.
	$A = \{A_0, \dots, A_{N_A-1}\}$	Set of predefined response actions.
	$\text{St}_m^{(l)}$	Compact RL state at step m in lifecycle l .
	$G_m^{(l)}$	Current attack stage at step m in lifecycle l .
	$F_m^{(l)}$	System artifact set at step m in lifecycle l .
	$Z_m^{(l)}$	System status at step m in lifecycle l .
	$R_m^{(l)}$	Reward received after a response action.
	$a_m^{(l)} \in A$	Action selected by the RL model at step m in lifecycle l .
	π_ϕ	RL response policy parameterized by ϕ .
	$\text{Prompt}^{(l)}$	LLM response prompt for lifecycle l .

3.2 Problem Statement

Modern SOC's receive alerts from multiple security tools, such as firewalls, IDS, NDR, and EDR. However, each alert usually describes only one part of an attack. Without correlation, defenders may know that suspicious activity exists, but still lack a clear view of the attacker's current behavior and the proper response. Therefore, the problem is not only attack detection, but also identifying the current MITRE ATT&CK technique and selecting a suitable mitigation action. The response must be strong enough to stop the lifecycle, but not so disruptive that it affects normal operations.

Figure 3.1 summarizes this problem. Multi-source alerts are grouped into an alert window, processed by the detection module, and converted into a detected technique. This technique is then used to build the response context, including the attack stage, affected artifacts, and system status. Finally, the response module selects a mitigation action from the predefined action set.

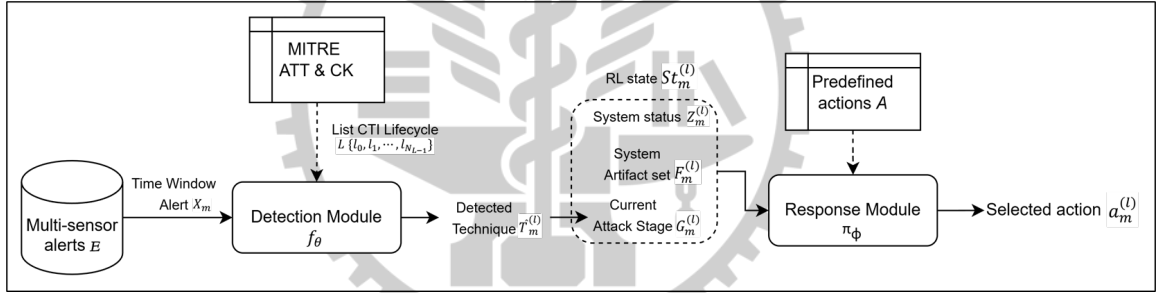


Figure 3.1: Overall problem formulation of the TRACE detection-response pipeline.

Formally, TRACE starts from a CTI-derived lifecycle set L , where each lifecycle l_i is an ordered sequence of MITRE ATT&CK techniques. During execution, alerts from FW, IDS, NDR, and EDR are grouped by timestamp into the alert window W_m . The multi-source alert set in this window is represented as $X_m = \{E_{t,R} \mid t \in W_m, R \in \{\text{FW}, \text{IDS}, \text{NDR}, \text{EDR}\}\}$. The detection model maps this alert set to the detected technique $\hat{T}_m^{(l)} = f_\theta(X_m)$, which provides the technique-level context for the current step of lifecycle l .

For response, the detected technique is used to derive the current attack stage $G_m^{(l)}$ and to support response-context construction. The compact RL state is represented as $\text{St}_m^{(l)} = (G_m^{(l)}, F_m^{(l)}, Z_m^{(l)})$, where $F_m^{(l)}$ is the system artifact set and $Z_m^{(l)}$ is the current system status. Based on this state, the response policy selects an action $a_m^{(l)} = \pi_\phi(\text{St}_m^{(l)})$, where $a_m^{(l)} \in A$. The selected action is

evaluated by how well it restores the system, supports suitable action selection across attack stages, and reduces the remaining attack lifecycle.

As shown in Figure 3.1, the detection and response modules are connected but solve different tasks. The detection module identifies the current MITRE ATT&CK technique from multi-source alerts. The response module uses the detected technique as response-context input, but the compact RL state is defined by the attack stage, system artifacts, and system status rather than by directly encoding the technique itself.

3.2.1 Technique Detection

Technique detection is formulated as the task of mapping a multi-source alert window to the attacker’s current MITRE ATT&CK technique. Given the alert set X_m in time window W_m , the detection model detects the current technique $\hat{T}_m^{(l)}$ for lifecycle l .

Input.

- Multi-source alert set X_m collected in time window W_m .
- CTI-derived lifecycle set $L = \{l_0, l_1, \dots, l_{N_L-1}\}$.

Output.

- Detected MITRE ATT&CK technique $\hat{T}_m^{(l)}$.

Objective. The objective is to learn a detection model f_θ that identifies the ground-truth technique $T_m^{(l)}$ from the alert evidence in X_m . The detection mapping is written as $\hat{T}_m^{(l)} = f_\theta(X_m)$, and the model is optimized to maximize the agreement between $\hat{T}_m^{(l)}$ and $T_m^{(l)}$. In this thesis, the detection task is evaluated using F1-score, accuracy, recall, and inference latency.

Constraints.

- The lifecycle set is finite and contains N_L available lifecycles.
- The alert evidence is collected from four sensor types: FW, IDS, NDR, and EDR.

3.2.2 Response Decision

Response decision is formulated as the task of selecting a mitigation action after a technique has been detected. At step m of lifecycle l , the detected technique $\hat{T}_m^{(l)}$ is used to derive the current attack stage $G_m^{(l)}$ and to construct the response context. The compact RL state is defined as $\text{St}_m^{(l)} = (G_m^{(l)}, F_m^{(l)}, Z_m^{(l)})$, where $G_m^{(l)}$ is the attack stage, $F_m^{(l)}$ is the relevant system artifact set, and $Z_m^{(l)}$ is the current system status.

Input.

- Detected technique $\hat{T}_m^{(l)}$, which is used to derive the response context.
- Current attack stage $G_m^{(l)} \in G$.
- System artifact set $F_m^{(l)}$.
- System status $Z_m^{(l)} \in Z$.
- Predefined action set $A = \{A_0, A_1, \dots, A_{N_A-1}\}$.

Output.

- Selected response action $a_m^{(l)} \in A$.

Objective. The objective is to learn a response policy π_ϕ that selects suitable mitigation actions for the current system state. The selected action should recover the compromised host, match the response strength to the current attack stage, and stop the attack lifecycle as early as possible. The response policy is written as $a_m^{(l)} = \pi_\phi(\text{St}_m^{(l)})$. In this thesis, this objective is evaluated using recovery success, suitable action selection across attack stages, and lifecycle stopping behavior.

Constraints.

- The selected action must belong to the predefined action set: $a_m^{(l)} \in A = \{A_0, A_1, \dots, A_{N_A-1}\}$.
- The system status must belong to $Z = \{1 : \text{NORMAL}, 2 : \text{COMPROMISED}\}$.
- The attack stage must belong to $G = \{1 : \text{EARLY}, 2 : \text{MID}, 3 : \text{LATE}\}$.

Chapter 4

TRACE: Transformer–RL Automated Cybersecurity Engine

This chapter presents TRACE: Transformer–RL Automated Cybersecurity Engine. TRACE combines a Transformer-based detection module and a reinforcement learning (RL)-based response module to support automated threat mitigation in SOC environments. The main goal of TRACE is to move from fragmented alert observation to actionable response. The detection module identifies the current MITRE ATT&CK technique from multi-source alerts, while the response module selects a mitigation action based on the attack stage, affected artifacts, and system status.

4.1 Solution Overview

TRACE is designed as a task-specific detection-and-response pipeline. Security alerts from firewall, IDS, NDR, and EDR systems are first correlated into a multi-source alert window. The Transformer detector converts this evidence into a technique-level attack context. The RL response module then uses this context to select a mitigation action from the predefined action space. TRACE is needed because detection alone cannot stop an attack lifecycle. A SOC must also decide how strongly to respond. Weak or delayed actions may allow the attack to continue, while overly aggressive actions may disrupt normal operations. TRACE addresses this by connecting fast technique detection with adaptive response decision-making.

A fully LLM-based pipeline can perform detection and response, but it may be too slow and computationally expensive for real-time SOC operation. Therefore, TRACE uses a fine-tuned Transformer for detection because security alerts are mostly textual or semi-structured, and Transformers can handle this type of security evidence better than classical machine learning

models while keeping lower latency than LLM inference.

RL is used because response selection is a sequential decision problem: each action changes the system state and can affect whether future techniques remain executable. The RL agent learns through trial and error and updates its policy to maximize reward. Different from general RL response methods, TRACE uses stage-aware decisions, so it can prefer lighter actions in early stages and stronger actions only when the attack reaches later stages.

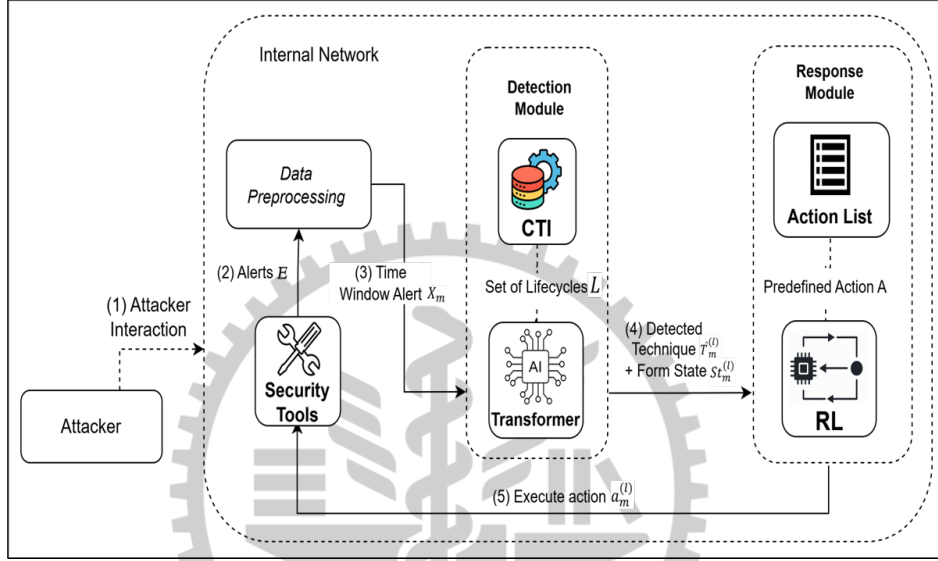


Figure 4.1: Overview of the TRACE detection-response framework.

4.2 Transformer-Based Technique Detection

The detection process starts by collecting alerts from multiple security tools in the SOC environment. As shown in Figure 4.2, firewall, IDS, NDR, and EDR systems generate events such as $E_{t,FW}$, $E_{t,IDS}$, $E_{t,NDR}$, and $E_{t,EDR}$ at time t . Each source observes a different part of the attack, including connection records, signature alerts, abnormal network behavior, endpoint processes, command lines, files, and user actions.

Before detection, alerts are normalized because different tools may describe similar information using different field names and formats. For example, a firewall log may use `src_ip`, `dst_ip`, and `dst_port`, while an IDS alert may use source address, destination port, and signature name. An EDR alert may focus on process name, command line, user account, and endpoint

activity. Normalization maps these heterogeneous fields into a common representation, making multi-source alerts easier to correlate and giving the model a consistent input format.

After normalization, alerts are ordered by timestamp to preserve the attacker's progression. A suspicious connection followed by command execution and service creation gives more useful context than the same events treated as unrelated alerts. TRACE then applies feature selection because not every raw field is useful for technique classification. Removing redundant, tool-specific, or administrative fields reduces noise and input length, which helps the detector focus on attack-related evidence and reduces detection latency.

After normalization, timestamp ordering, and feature selection, TRACE groups the selected events into time windows. For each time window W_m , TRACE constructs a multi-source alert set X_m from available sensor events. The detection mapping is written as $\hat{T}_m^{(l)} = f_\theta(X_m)$, where f_θ is the Transformer-based detector and $\hat{T}_m^{(l)}$ is the detected technique at step m of lifecycle l . The technique-level output gives the response module a clearer description of attacker behavior than isolated raw alerts, allowing the system to reason about which ATT&CK technique is currently being executed and use that context for mitigation.

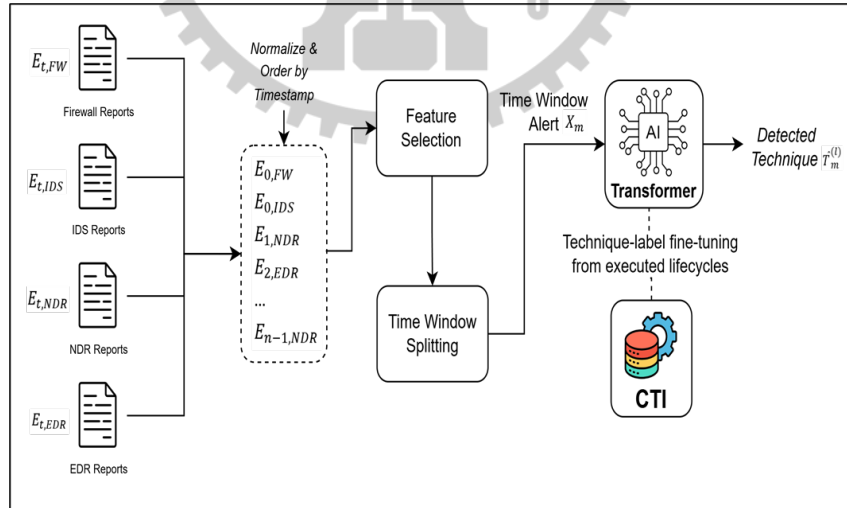


Figure 4.2: Transformer-based technique detection process.

4.3 RL-Based Response Decision

After the current technique is detected, the response module selects a mitigation action. The objective is to restore the host, reduce future attack progress, and adjust response strength according to the current attack stage.

TRACE uses the detected technique as input to construct the response context, but the technique label itself is not directly used as the RL state. The compact RL state is written as $\text{St}_m^{(l)} = (G_m^{(l)}, F_m^{(l)}, Z_m^{(l)})$, where $G_m^{(l)}$ is the current attack stage, $F_m^{(l)}$ is the affected artifact set, and $Z_m^{(l)}$ is the host status. The stage controls response strength, the artifact set supports targeted remediation, and the status indicates whether the host is NORMAL or COMPROMISED.

4.3.1 Response Action Space

The response action space contains mitigation actions grouped by operational impact level. Some actions are adapted from prior RL-based intrusion response work [20]. However, the action set is extended in this thesis because the reference actions alone are not sufficient to stop all attack behaviors in the experiment. Additional actions are designed from the executed MITRE ATT&CK and Metasploit-based attack procedures, such as service and scheduled task cleanup, restricted command execution, and restricted write operations.

Low-impact actions mainly focus on targeted cleanup or limited intervention. Examples include process cleanup, file cleanup, registry cleanup, service/task cleanup, force sign out, and TCP connection reset. These actions are useful when the system can recover by removing specific artifacts. However, low-impact actions may not always reduce the attacker's capability enough. If the attacker still has credentials, execution capability, or network access, the lifecycle may continue to the next technique.

Medium-impact actions are designed to reduce the attacker's capability while avoiding full endpoint disruption. Examples include force password reset, disabling a user account, adding an IP address to a block list, restricting command execution, and restricting write operations.

These actions can limit the attacker’s ability to execute commands, create persistence, modify files or registry values, and communicate with external infrastructure.

High-impact actions provide stronger containment, such as endpoint isolation and workstation shutdown. These actions can stop attacker activity more aggressively, but they also have higher operational cost because they may interrupt normal services or disconnect a host from the network. Therefore, high-impact actions should be used more carefully and are more suitable when the attack has progressed to a later stage or when lighter actions are insufficient.

This grouping allows the agent to learn when lightweight recovery is sufficient and when stronger containment is necessary. Therefore, the policy can balance recovery effectiveness and operational cost instead of always selecting the strongest action.

Table 4.1: Response Action Space Grouped by Operational Impact Level

Impact Level	Action Name	Description
Low	Force sign out	Logs out a user from active sessions to stop unauthorized session use.
	Process cleanup	Terminates malicious process artifacts.
	File cleanup	Removes malicious files and staged collection artifacts.
	Registry cleanup	Reverts malicious registry key changes.
	Service / Task cleanup	Removes malicious service and scheduled task artifacts.
	Reset TCP connection	Drops or resets a suspicious network connection.
Medium	Force password reset	Forces a user to reset the password to mitigate credential compromise.
	Disable user account	Disables a user account to stop unauthorized access.
	Add IP to block list	Blocks suspicious network communication through a firewall rule.
	Restrict command execution	Blocks or limits command interpreters and scripting tools such as cmd.exe, PowerShell, and WMIC.
	Restrict write operations	Blocks or limits file writes, registry edits, service creation, and scheduled task creation.
High	Shutdown workstation	Shuts down an asset to stop activity, with high service disruption.
	Isolate endpoint	Disconnects an asset from the network to contain the attack.

4.3.2 Reward Function

The reward function guides the RL agent toward practical response behavior. TRACE uses three reward components: outcome reward, action reward/penalty, and future technique suppression reward. These components are designed to balance immediate host recovery, response

cost, and long-term lifecycle stopping.

The outcome reward evaluates whether the selected response actions restore the host. If the host becomes NORMAL, the agent receives a positive reward because the response successfully removes the active compromise condition. If the host remains COMPROMISED, the agent receives a penalty because the selected actions fail to recover the system. This component gives the agent direct feedback on the main response objective.

The future technique suppression reward encourages the agent to stop not only the current technique, but also later techniques in the same lifecycle. This is important because a response may appear successful in the current step but still leave the attacker able to continue. By rewarding blocked future techniques, the agent learns to prefer actions that reduce the attacker's future options. The reward is capped to avoid over-rewarding overly aggressive actions, so the agent does not always choose high-impact containment only to gain suppression reward.

The action reward/penalty controls the operational impact of each response action. In the stage-aware setting, low-impact actions are preferred in the early stage, medium-impact actions are encouraged in the mid stage, and stronger actions are allowed in the late stage. This reflects the practical SOC idea that response strength should increase as the attack progresses. In the risk-aware comparison setting, the model considers action impact but does not use stage information, so it mainly favors lower-impact actions regardless of attack progression.

Together, these reward components help the RL agent learn a balanced response policy. The agent is rewarded for recovery, encouraged to reduce future attack progress, and penalized when the selected action is too disruptive for the current stage.

Table 4.2: Combined Reward Design for the TRACE Response Agent

Reward Component	Setting	Condition / Stage	Reward			Meaning
			Low Impact	Medium Impact	High Impact	
Outcome Reward	Stage-aware & Risk-aware	NORMAL	+2.0			System restored successfully.
		COMPROMISED	-1.5			System remains compromised.
Future Technique Suppression Reward		Blocked Future Techniques	$\min(b_{\text{blocked}} \times 1, 6)$			Rewards future blocking but limits high-impact over-reward.
Action Reward / Penalty	Stage-Aware	Early	+0.5	0.0	-2.0	Prefer low-impact actions early.
		Mid	0.0	+0.5	-1.0	Prefer medium-impact actions in mid stage.
		Late	-0.5	+0.5	+1.0	Allow stronger actions late.
	Risk-Aware	All Stages	+0.5	0.0	-1.5	Prefer low-impact actions and penalize high-impact actions.

4.3.3 Stage-Aware Response Design

Stage-aware response design is used because the same mitigation action can have different value depending on attack progress. In the early stage, the attacker may only be entering the system or starting execution, so lightweight actions such as connection reset, process cleanup, or artifact cleanup are often preferable. In the late stage, the attacker may already perform discovery, lateral movement, collection, command and control, exfiltration, or impact actions. At this point, stronger containment may be more reasonable because the risk of continuing the lifecycle is higher.

TRACE defines the current stage using two configurations for comparison. The first configuration is based on technique progress, where techniques 1–3 are treated as Early, techniques 4–6 as Mid, and techniques 7 and above as Late. The second configuration is based on MITRE ATT&CK tactics. This tactic-based setting gives the stage a semantic meaning: Early represents initial entry and execution, Mid represents maintaining access and avoiding detection, and Late represents expansion, data theft, command and control, and final impact.

The stage value is used by the reward function and the learned policy to control response strength. This allows TRACE to prefer less disruptive actions when the attack is still early, while allowing stronger actions when the attack reaches later and more damaging phases. Table 4.3 summarizes the stage definitions used in the stage-aware response design.

Table 4.3: Stage Definition Used in the Stage-Aware Response Design

Stage	Technique-Progress Rule	Tactic-Based Definition	Response Intention
Early	1–3	Reconnaissance Initial Access Execution	Prefer low-impact actions to avoid unnecessary disruption.
Mid	4–6	Persistence Privilege Escalation Defense Evasion Credential Access	Limit attacker capability while preserving system operation.
Late	≥ 7	Discovery Lateral Movement Collection Command & Control Exfiltration Impact	Allow stronger containment because lifecycle risk is higher.

4.3.4 Stage-Aware DDQN Response Algorithm

TRACE trains the response agent using a stage-aware Double Deep Q-Network (DDQN). The online Q-network selects actions, the target Q-network stabilizes target estimation, and the replay buffer stores transitions for mini-batch learning. The trained policy maps the compact response state to a mitigation action.

Each episode samples one attack lifecycle. For each technique, the agent can attempt at most $K_{\max} = 3$ actions while the host remains COMPROMISED. The agent selects actions using an ϵ -greedy strategy, executes the selected action, observes the next state, and updates the DDQN policy.

The reward calculation follows three levels. First, the stage-aware or risk-aware action reward is calculated inside the per-technique response loop. Second, the host outcome reward is calculated after the response attempts for the current technique. Third, the future technique suppression reward is accumulated when the lifecycle finishes. This structure helps the agent learn both immediate recovery and long-term lifecycle stopping behavior.

Algorithm 4.1 Simplified Stage-Aware DDQN Response Agent

Inputs: lifecycle set L , action set A , discount factor γ , exploration rate ϵ , maximum attempts $K_{\max} = 3$, episodes N_e

Output: trained response policy $\pi_\phi(s)$

- 1: Initialize online network Q_θ , target network $Q_{\theta-}$, and replay buffer D
 - 2: **for** episode = 1 to N_e **do**
 - 3: Sample a lifecycle $l \in L$ and reset the environment
 - 4: Set blocked-technique counter $b_{\text{blocked}} \leftarrow 0$
 - 5: **for** each technique step m in lifecycle l **do**
 - 6: Receive detected technique $\hat{T}_m^{(l)}$
 - 7: Build state $s_t \leftarrow (G_m^{(l)}, F_m^{(l)}, Z_m^{(l)})$ and set $k \leftarrow 0$
 - 8: **while** $Z_m^{(l)} = \text{COMPROMISED}$ **and** $k < K_{\max}$ **do**
 - 9: Select action a_t using ϵ -greedy DDQN
 - 10: Execute a_t and observe s_{t+1} , updated status $Z_m^{(l)}$, and newly blocked techniques Δb
 - 11: $b_{\text{blocked}} \leftarrow b_{\text{blocked}} + \Delta b$
 - 12: *Reward 1: stage-aware action reward*
 - 13: $r_{\text{action}} \leftarrow \text{StageReward}(G_m^{(l)}, \text{Impact}(a_t))$
 - 14: Store $(s_t, a_t, r_{\text{action}}, s_{t+1})$ in D
 - 15: $s_t \leftarrow s_{t+1}$; update ϵ ; $k \leftarrow k + 1$
 - 16: *Reward 2: host recovery outcome reward*
 - 17: Add $r_{\text{outcome}} \leftarrow \text{OutcomeReward}(Z_m^{(l)})$ to the final transition of the current technique
 - 18: Update Q_θ using DDQN replay with γ and periodically copy Q_θ to $Q_{\theta-}$
 - 19: *Reward 3: future technique suppression reward*
 - 20: Add $r_{\text{stop}} \leftarrow \min(b_{\text{blocked}}, 6)$ to the final lifecycle transition
 - 21: **end for**
 - 22: **end for**
 - 23: **return** trained policy $\pi_\phi(s)$
-

4.4 Example Run

Figure 4.3 illustrates a simplified TRACE run using one attack lifecycle. The lifecycle includes T1059 Command and Scripting Interpreter (Execution), T1543 Create or Modify System Process (Persistence), T1112 Modify Registry (Privilege Escalation), T1027 Obfuscated Files or Information (Defense Evasion), and T1105 Ingress Tool Transfer (Command and Control).

Security telemetry from Firewall, IDS, NDR, and EDR is first mapped to MITRE ATT&CK techniques and then converted into response context for the RL response agent. TRACE first detects T1059 execution behavior and selects process cleanup to remove malicious execution artifacts. The lifecycle then progresses to T1543 persistence, where TRACE applies file cleanup and service cleanup to remove persistence-related artifacts. When T1112 registry modification is detected, the RL agent selects endpoint isolation because the attack has entered a later stage, where the compromise risk is higher and stronger containment is needed.

After endpoint isolation, the lifecycle is interrupted before T1027 defense evasion and T1105 command-and-control payload transfer can occur. This example demonstrates that TRACE does not apply a single fixed response to all alerts; instead, it adapts mitigation actions according to the detected technique, attack stage, and affected system artifacts to stop the attack before further progression.

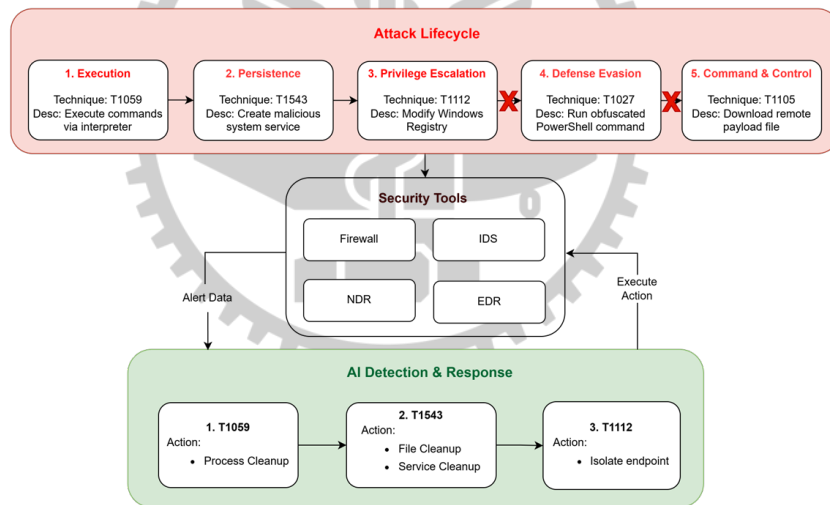


Figure 4.3: Example run of TRACE from attack lifecycle detection to response action selection.

Chapter 5

Implementation

This chapter describes how TRACE is implemented in the experimental SOC testbed. The implementation is organized around three practical aspects: the open-source tools and libraries used to build the cyber range, the hardware environment used for detection and response experiments, and the workflow that connects attack execution, alert preprocessing, technique detection and response-state construction.

5.1 Open-Source Tools and Libraries

The TRACE implementation uses a toolchain that supports the full experimental flow from attack scenario construction to response evaluation. VMware Workstation is used as the virtualization layer, so the attacker host, victim host, SOC components, and security tools can run in an isolated cyber range. This isolation is important because the experiments execute real attack behaviors and response actions, but they must not affect the external network or production systems.

The security tools provide the multi-source telemetry used by TRACE. OPNsense acts as the firewall and records gateway-level traffic evidence, such as allowed or suspicious connections. Suricata is used as the IDS to detect packet- and signature-based network attacks. Wazuh is then used to streamline both network and endpoint evidence: Wazuh with Suricata supports NDR-style network monitoring, while Wazuh with Sysmon provides EDR evidence from the victim host. Through this combination, TRACE can observe both network behavior and host behavior, which is necessary because some techniques are more visible in traffic logs, while others are only clear from process, command-line, file, registry, or service activity.

Attack execution and scenario construction are supported by Atomic Red Team, Metasploit, and MITREtrieval. MITREtrieval provides CTI-derived attack lifecycles, which define the ordered techniques used in the experiment. Atomic Red Team and Metasploit then execute representative ATT&CK behaviors inside the cyber range. The learning and analysis components are implemented with Python libraries. Python scripts are used to normalize alerts, select sighting features, split events into time windows, construct response states, train the response agent, and process evaluation logs. PyTorch supports the learning components, while Matplotlib and Seaborn are used to generate evaluation figures and analyze experiment results. Table 5.1 summarizes the main tools and libraries used in the implementation.

Table 5.1: Open-Source Tools and Libraries Used in the Implementation

Category	Name	Description and Functionality
Virtualization Tools	VMware Workstation	Runs isolated virtual environments for testing.
Security Tools	OPNsense (Firewall)	Filters traffic and applies network rules.
	Suricata (IDS)	Detects malicious network traffic and generates IDS alerts.
	Wazuh + Suricata (NDR)	Analyzes network flows and network alerts for threats.
	Wazuh + Sysmon (EDR)	Monitors endpoints and generates host-level security alerts.
Attacking Tools	Atomic Red Team + Metasploit	Simulates MITRE ATT&CK techniques in the testbed.
CTI Resource	MITREtrieval	Provides CTI-derived lifecycles for attack scenario construction.
Python Libraries	PyTorch	Implements and trains the RL algorithm.
	Matplotlib	Visualizes metrics and experimental results.
	Seaborn	Creates statistical and correlation plots.

5.2 Testbed Setup

The experiments are executed on two machines because the detection module requires a stronger computing environment than the response module. The detection experiments use a dedicated GPU server equipped with an Intel Xeon w5-3435X processor with 16 physical cores and 32 logical threads. The server contains two NVIDIA GeForce RTX 4090 GPUs, each with approximately 23.64 GB of VRAM. This environment is used for Transformer-based technique detection and detection-model evaluation.

The response experiments use a separate response machine with 6 physical CPU cores and 12 logical threads, together with an NVIDIA GeForce RTX 3060 GPU with 12 GB of VRAM. This lighter environment is sufficient for evaluating the RL response module because response-state construction and action selection require less computation than Transformer model evaluation. Therefore, the testbed uses a higher-capacity machine for detection and a separate lower-cost machine for response evaluation.

5.3 Implementation Details

Figure 5.1 shows the implemented TRACE workflow from attack execution to response feedback. The workflow starts with CTI-derived lifecycle examples produced by MITREtrieval. These lifecycles guide the attacker component, where Atomic Red Team and Metasploit execute ATT&CK techniques against the user host in the cyber range. During execution, the security tools collect evidence from different viewpoints, including firewall, IDS, NDR, and EDR telemetry.

The collected telemetry is passed to the data preprocessing component, which normalizes alert fields, orders events by timestamp, and selects useful features. The processed evidence is then sent to the Transformer-based detection module to identify the current MITRE ATT&CK technique. After detection, the technique output and current system evidence are used to form the response context for the RL response module. The RL agent selects a mitigation action from the predefined action list and executes it through the available action targets, such as the user host or security-tool APIs. Finally, the environment checks the result of the action and returns feedback for response learning.

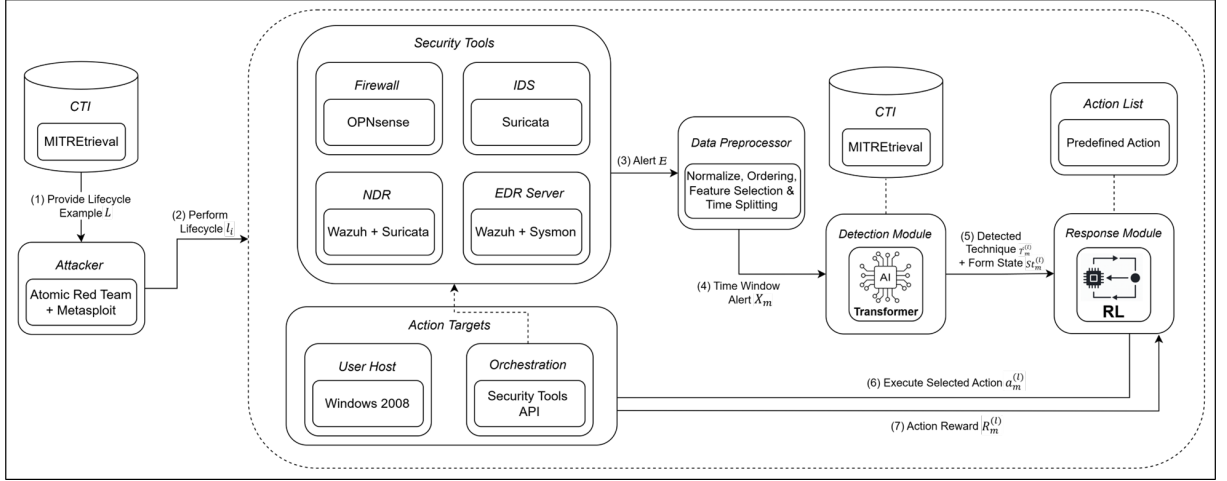


Figure 5.1: Implemented system architecture of the TRACE detection-and-response pipeline.

5.3.1 Sighting Feature Construction

The detection module uses sightings as its input representation. A sighting is a time-windowed group of security alerts collected from multiple tools. Instead of using one raw alert at a time, TRACE combines network and host evidence so that the detection model can observe a more complete view of the attack. Network evidence describes communication behavior, such as protocol usage, destination ports, signatures, and severity. Host evidence describes endpoint behavior, such as rule descriptions and command-line activity.

Table 5.2 summarizes the selected sighting features from each security tool. Only useful fields are retained because many raw alert fields are identifiers, timestamps, or metadata that do not directly describe attacker behavior. Removing these unnecessary fields reduces noise and keeps the detection input shorter, which is important for maintaining low inference latency.

Table 5.2: Selected Sighting Features from Security Tools

Feature Type	Security Tool	Sighting Feature	Description
Network	Firewall	proto	Network protocol used.
	Firewall	dst_port	Destination port of the connection.
	IDS	proto	Network protocol used.
	IDS	dst_port	Destination port of the connection.
	NDR	signature	Detected attack pattern.
	NDR	severity	Threat severity level.
Host	EDR	rule_desc	Detection rule triggered on the host.
	EDR	command_line	Executed command on the host.

5.3.2 Response-State Implementation

After the detection module identifies the current technique, the response module constructs a compact RL state. The state contains three main types of information: current stage, system artifact state, and system status. The current stage provides attack-progress information, the artifact state records the remaining evidence of compromise, and the system status indicates whether the host has returned to normal.

System Artifact State

The system artifact state captures compromise-related evidence that remains on the host or in the network. These artifacts are defined from behaviors observed during Atomic Red Team and Metasploit execution. The implementation tracks process, network, account, scheduler, service, file, and registry states so that the response agent can identify which part of the system still needs recovery. For example, a suspicious process may require process termination, while a suspicious connection may require a TCP reset or IP blocking.

A practical example is shown in Figure 5.2. Most artifacts are normal, but the network state contains an active connection from 192.168.81.6 to 10.0.0.5, which represents a possible command-and-control connection. The registry state is also compromised because `UseLogonCredential` is expected to be 0, but the observed value is 1. These artifact values are used to form the RL state, so the agent receives concrete evidence about which parts of the system are still unsafe. This helps the agent select targeted actions, such as resetting the suspicious connection, blocking the destination IP, or restoring the unsafe registry value.

Table 5.3: RL System Artifact State Representation

Artifact Field	Example Fields Inside
Process State	suspicious_processes
Network State	active_connections
Account State	account_artifacts
Scheduler State	scheduler_artifacts
Service State	service_artifacts
File State	file_created, modification_file, deleted_file
Registry State	registry_artifacts, e.g., UseLogonCredential

System Status

The system status indicates whether the host is in a NORMAL or COMPROMISED state. This status is determined by comparing the current artifact state with the baseline normal state. If all related artifacts are removed, cleared, or restored to baseline values, the host is considered NORMAL. If any relevant artifact still exists, the host remains COMPROMISED. This status is used for both reward calculation and evaluation because it directly represents whether the response action successfully restored the host.

Using the example in Figure 5.2, the host is marked as COMPROMISED because the network and registry artifacts still deviate from the normal baseline. Even if the process, account, scheduler, service, and file states are normal, one remaining command-and-control connection or unsafe registry setting is enough to indicate that the host is still COMPROMISED. After the selected response action removes the suspicious connection and restores the registry value to the normal baseline, the system status can be updated to NORMAL.

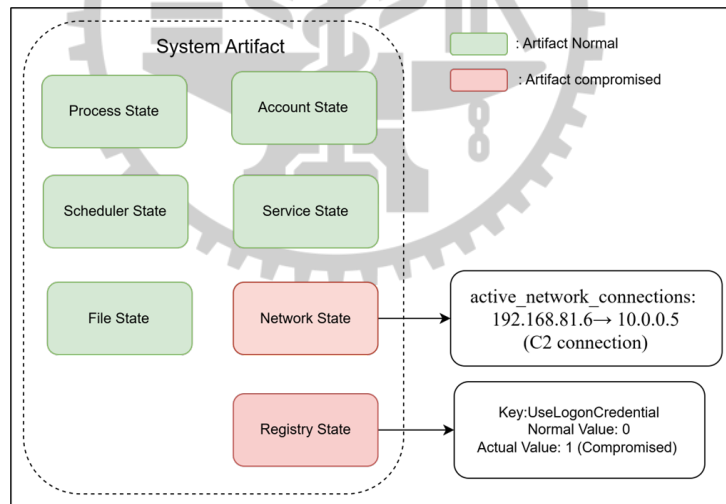


Figure 5.2: System status determination from artifact states.

Chapter 6

Evaluation and Results

This chapter evaluates the effectiveness of TRACE from four perspectives. First, it defines the experimental baselines and model configurations used for detection and response. Second, it profiles the dataset, lifecycle distribution, and sighting statistics. Third, it evaluates technique detection, RL response learning, stage-aware response behavior, and lifecycle stopping. Finally, it compares the end-to-end latency of the TRACE Transformer-based detection and RL-based response pipeline against LLM-based detection–response pipelines.

6.1 Baseline

This section defines the experimental baseline configurations used in the evaluation. The baselines are organized into three groups: technique detection models, response decision models, and the LLM-based detection–response pipeline. The purpose of this section is to specify the model settings, training parameters, and response baseline configuration used for comparison.

6.1.1 Detection Model Configuration

Table 6.1 summarizes the configuration used for the detection baselines. All models use the same 80% training and 20% testing split to keep the comparison consistent. Both Transformer and LLM baselines use a maximum sequence length of 512, but their training settings are different. The Transformer models are trained for 10 epochs with a learning rate of 2×10^{-5} and gradient accumulation of 2, while the LLM baselines are trained for 5 epochs with a learning rate of 2×10^{-4} and gradient accumulation of 32. These settings reflect the different training requirements between compact Transformer models and larger LLM-based models.

The Transformer candidates include DistilRoBERTa, SecureBERT, and SecRoBERTa, while the LLM candidates include LLaMA 3 Instruct, Gemma 3, and Foundation-Sec. The Trans-

former configuration is designed for efficient fine-tuning and low-latency inference, whereas the LLM configuration requires larger gradient accumulation to support training with larger model sizes and higher memory demand. Therefore, this configuration allows the evaluation to compare not only model performance, but also the practical cost difference between lightweight Transformer-based detection and LLM-based detection.

Table 6.1: Transformer and LLM Detection Model Configuration

Item	Transformer	LLM
Train ratio	80% train / 20% test	80% train / 20% test
Model candidates	DistilRoBERTa, SecureBERT, SecRoBERTa	LLaMA 3 Instruct, Gemma 3, Foundation-Sec
Max sequence length	512	512
Number of epochs	10	5
Learning rate	2×10^{-5}	2×10^{-4}
Gradient accumulation	2	32

6.1.2 Response Model Configuration

For response evaluation, the DDQN agent is trained for 12,000 episodes. The same response model configuration is used across the three response settings: No Stage, Technique Progress, and Tactic. This keeps the comparison focused on the effect of stage design rather than differences in training parameters.

Table 6.2 summarizes the RL response model configuration. The agent uses DDQN as the learning algorithm, with an estimated 240,000 training timesteps. The learning rate is set to 5×10^{-5} , and the batch size is 256 to support stable updates during training. The exploration fraction is 0.15, with the final exploration epsilon reduced to 0.02, allowing the agent to explore different response actions early and gradually rely more on the learned policy. The discount factor is set to 0.99 so that the agent considers both immediate recovery and future lifecycle suppression. The network architecture uses two hidden layers with sizes [256, 256], which keeps the response policy lightweight while still providing enough capacity for action selection.

Table 6.2: RL Response Model Configuration

Item	Setting
RL algorithm	DDQN (Double Deep Q-Network)
Training episodes	12,000
Estimated timesteps	240,000
Learning rate	5×10^{-5}
Batch size	256
Exploration fraction	0.15
Final exploration epsilon	0.02
Discount factor	0.99
Gradient steps	4
Network architecture	[256, 256]

6.1.3 LLM Detection and Response Baseline

The LLM-based pipeline is used as a comparison baseline for the complete detection–response process. In this pipeline, LLM-based models are used for both detecting the current technique and selecting the response action. The CTI-fine-tuned detection model first receives alert evidence and detects the current MITRE ATT&CK technique. The detection model configuration for the LLM baseline is summarized in Table 6.1.

Figure 6.1 illustrates the LLM-based detection–response pipeline. After the current technique is detected, the response context is constructed and inserted into a response prompt. The response model then selects an action from the available response actions. Although this pipeline provides flexibility for both technique detection and response decision-making, it may introduce higher end-to-end latency because LLM inference is generally slower.

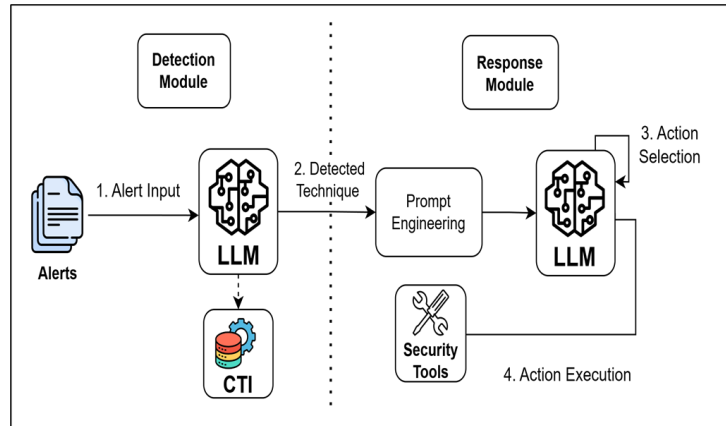


Figure 6.1: LLM-based detection and response pipeline.

Table 6.3 summarizes the LLM response model configuration. The LLM response models are accessed through OpenRouter and are used to evaluate prompt-based action selection against the trained RL response policy.

Table 6.3: LLM Response Model Configuration

Item	Setting
Backend/API	OpenRouter
Default models	openai/gpt-4.1-mini; google/gemini-2.5-flash
Timeout	120 seconds

For the response baseline, the LLM receives the same major response context used by the RL agent, including the current attack stage, system artifacts, system status, and available response actions. The prompt also follows the same response principle: prefer lower-impact actions in early or less severe situations, allow stronger actions in later or more severe situations, and restore the system while avoiding unnecessary disruption.

Figure 6.2 shows an example response prompt. In this example, the attack is in the Late stage, the system status is COMPROMISED, and the artifacts include a suspicious process, malicious registry key, and active outbound connection to a suspicious IP. The LLM is given the available actions and asked to choose the best action for stopping the attack and recovering the system.

Example of LLM Response Prompt

```
You are a cybersecurity response agent.

Current stage: Late
System status: COMPROMISED
Artifacts:
- suspicious process detected
- malicious registry key exists
- active outbound connection to suspicious IP

Available actions:
- Process cleanup
- Registry cleanup
- Reset TCP connection
- Isolate endpoint
- Shutdown workstation

Choose the best action to stop the attack and recover the system.
Prefer lower-impact actions when possible, but use stronger actions if the attack is severe.
```

Figure 6.2: Example of LLM response prompt design.

6.2 Data Profiling

This section profiles the dataset and lifecycle coverage used in the experiments. The data are generated from CTI-derived attack lifecycles and implemented attack scenarios. During attack execution, security alerts are collected from Firewall, IDS, NDR, and EDR sources, normalized by timestamp, grouped into time-windowed alert inputs, and compared against the ground-truth MITRE ATT&CK technique labels.

Figure 6.3 shows the lifecycle length distribution of CTI-derived attack scenarios. Most lifecycles contain approximately five to ten techniques, with an average length of about 8.6 techniques and a maximum length of 19 techniques. This distribution is important because longer lifecycles create more opportunities for the response module to stop the attack before it reaches later stages.

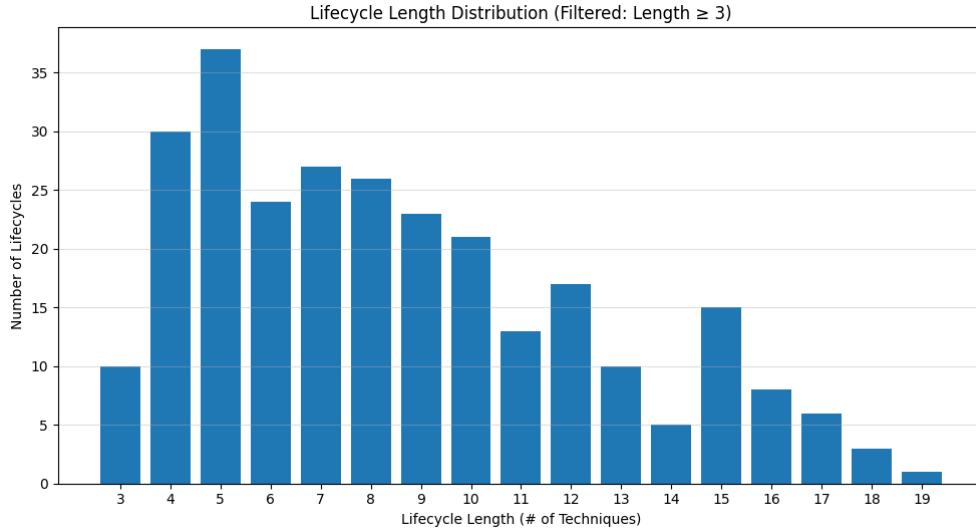


Figure 6.3: Lifecycle length distribution of CTI-derived attack scenarios.

Table 6.4 summarizes the collected sighting statistics. In total, the implementation collected 13,663 alerts. Each time window contains 2.13 alerts on average, with a range from one to eight alerts per window. EDR contributes the largest number of alerts because many ATT&CK techniques leave host-level traces such as process execution, command-line activity, file modification, or registry changes. IDS, firewall, and NDR provide complementary network evidence.

Table 6.4: Collected Sighting Statistics

Item	Value
Total collected alerts	13,663
Average alerts per window	2.13
Alert range per window	1 to 8
EDR alerts	10,822
IDS alerts	1,521
Firewall alerts	828
NDR alerts	492

6.3 Technique Detection Performance

The first result evaluates whether the detection module can identify the current MITRE ATT&CK technique from multi-source alert evidence. The compared models include LLM-based detectors and Transformer-based detectors. The evaluation considers F1-score, accuracy, recall, and inference latency, because the detector must provide both reliable technique labels and fast out-

puts for the response module.

Figure 6.4 shows that the Transformer-based detectors achieve similar or slightly better detection quality than the LLM baselines. SecBERT obtains the highest F1-score of 0.866, while SecRoBERTa reaches 0.856. In comparison, the LLM baselines obtain F1-scores of 0.833–0.834. The accuracy and recall values are also close across the compared models, which indicates that compact Transformer models can still capture enough textual and contextual information from the alert evidence to classify the current technique.

The main difference is latency. The Transformer-based detectors require only 2.78–5.76 ms/sample, while the LLM-based detectors require approximately 62.29–62.35 ms/sample. This latency gap is important for SOC operation because the detected technique is needed before the response state and mitigation action can be constructed. Therefore, even when detection accuracy is similar, the lower inference latency makes the Transformer-based detector more suitable for real-time threat mitigation.

Overall, this result supports using a specialized Transformer model for technique detection instead of relying only on LLM inference. The Transformer models provide competitive detection performance, but with much lower runtime cost. This makes the detection module fast enough to support the next response stage in TRACE.

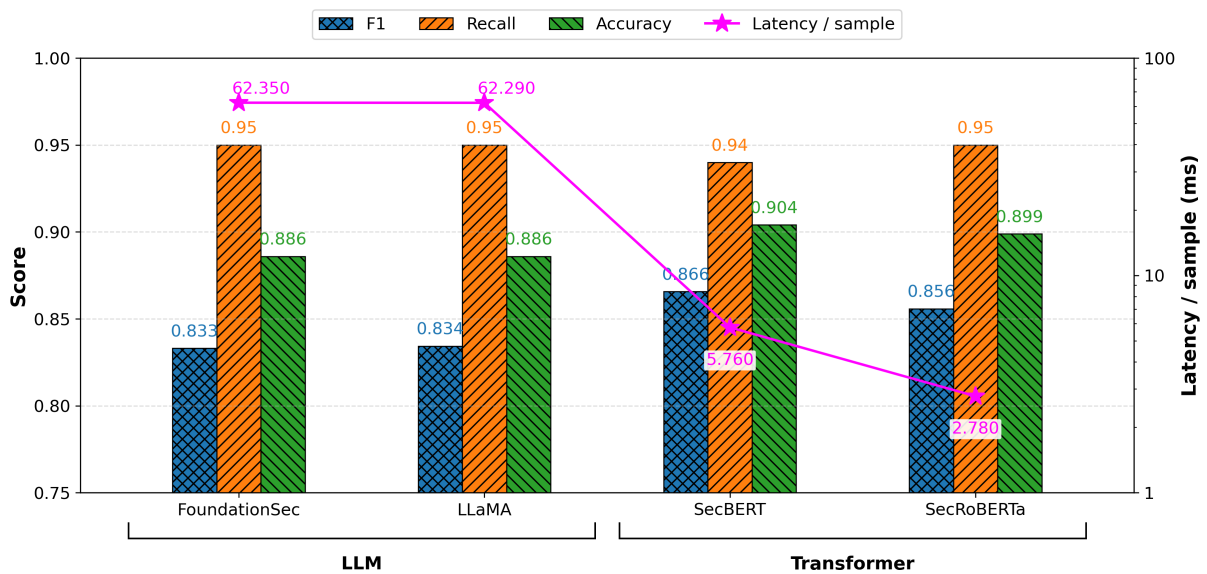


Figure 6.4: LLM and Transformer performance in MITRE ATT&CK technique detection.

6.4 RL Response Decision

The second result evaluates the RL response agent. This evaluation focuses on three questions: whether the agent learns to recover compromised hosts, whether the stage-aware design changes action strength across attack stages, and whether the selected actions stop attack life-cycles earlier.

6.4.1 Recovery Learning Curve

Figure 6.5 shows the system recovery success rate during training. At the beginning, the success rate is only about 40%–50%. This is expected because the agent is still exploring the action space and has not yet learned which actions can remove the compromised artifacts in each state.

After training progresses, the recovery success rate increases and becomes stable. After around 9,000 episodes, the success rate remains above 90% across the evaluated settings. This trend shows that the DDQN agent learns a stable recovery policy from environment feedback. Instead of applying fixed rules, the agent learns which actions are more likely to restore the host to a normal state under different artifact and status conditions.

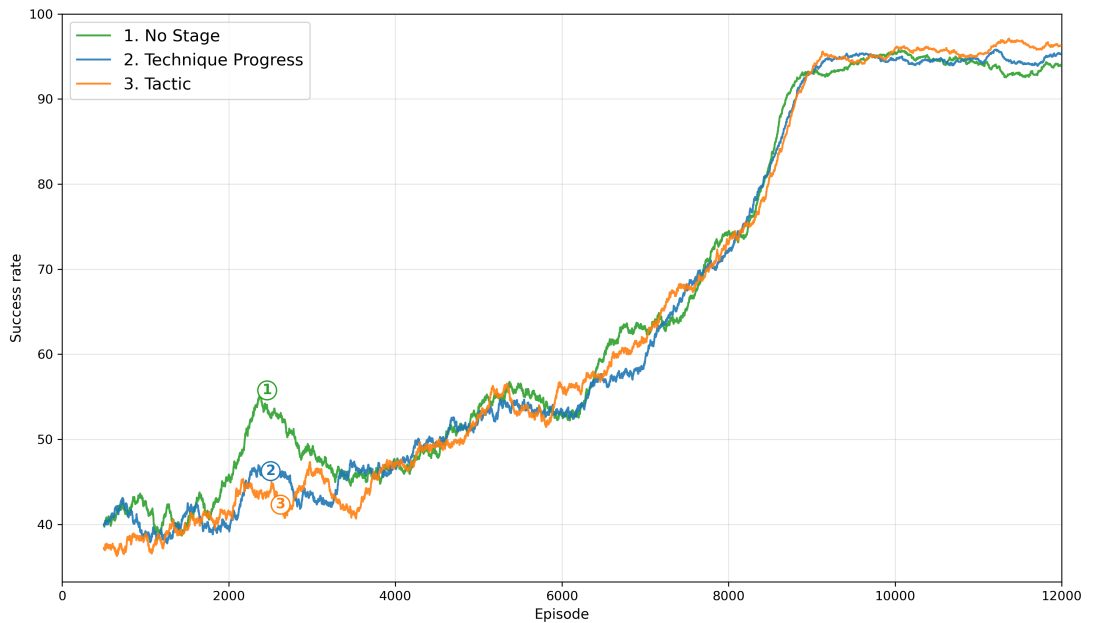


Figure 6.5: RL learning curve of system recovery success rate per episode.

6.4.2 Stage-Aware Action Selection

Figure 6.6 shows the updated action-impact distribution across Early, Mid, and Late stages. In the Early stage, all configurations mostly select low-impact actions. No Stage selects 91.9% low-impact actions, Technique Progress selects 92.7%, and Tactic selects 90.7%. High-impact actions remain very rare in the Early stage, with only 0.6%, 0.9%, and 1.2%, respectively. This indicates that the learned policy generally avoids aggressive containment when the attack is still at an early point in the lifecycle.

In the Mid stage, the three configurations show different response tendencies. No Stage remains mostly low impact, with 82.9% low-impact actions and 16.1% medium-impact actions. Technique Progress becomes more balanced, with 73.6% low-impact, 18.3% medium-impact, and 8.1% high-impact actions. In contrast, Tactic remains the most conservative in the Mid stage, selecting 93.9% low-impact actions, 3.4% medium-impact actions, and 2.7% high-impact actions. This shows that the stage definition affects when the RL agent begins to shift from cleanup-oriented actions to stronger mitigation.

The main difference appears in the Late stage. No Stage still selects 88.0% low-impact actions and only 1.2% high-impact actions, which shows that the risk-aware-only setting remains conservative even when the attack has progressed. In contrast, the stage-aware settings increase stronger actions in the Late stage. Technique Progress selects 29.2% high-impact actions, while Tactic selects 28.0% high-impact actions and 28.1% medium-impact actions. This means that stage-aware reward design allows the agent to escalate response strength when the lifecycle reaches a more serious stage.

Overall, the result shows that stage-aware reward design changes response behavior according to attack progression. Early-stage actions remain mostly low impact to reduce unnecessary disruption. In the Mid stage, Technique Progress begins to introduce more medium- and high-impact actions, while Tactic remains more conservative. In the Late stage, both stage-aware configurations allow stronger mitigation than No Stage. This behavior is important because stronger actions become more acceptable when the lifecycle reaches a later and riskier stage.

Therefore, stage-aware RL provides more practical response behavior by remaining conservative early while becoming stronger when late-stage mitigation is needed.

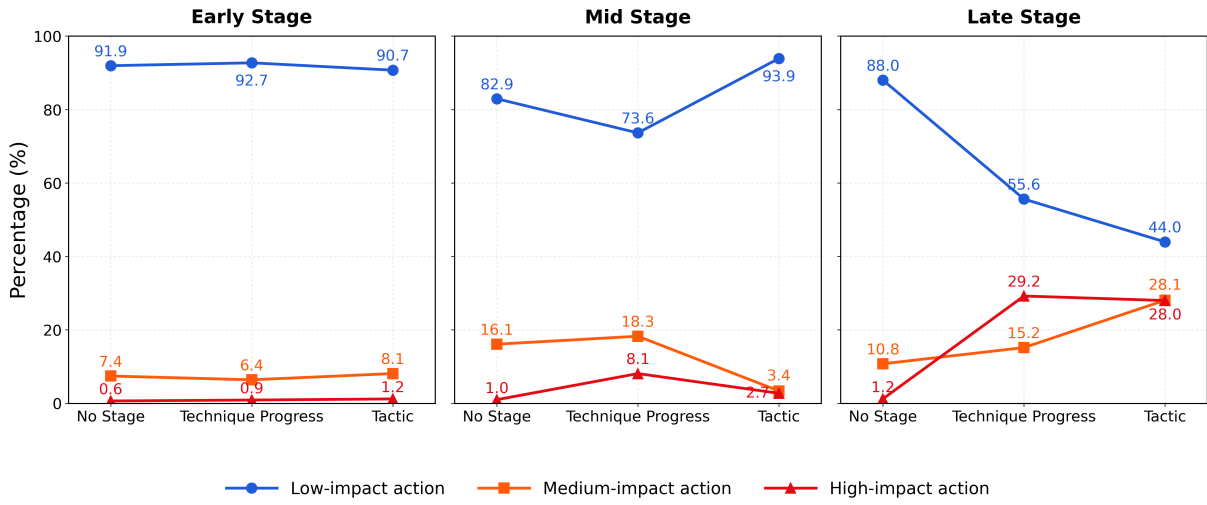


Figure 6.6: RL action-impact distribution across attack stages.

6.4.3 Lifecycle Stopping Behavior

Figure 6.7 compares where the attack lifecycle stops under No Stage, Technique Progress, and Tactic. Discovery is an important stopping point because stopping the lifecycle at this tactic can prevent the attacker from moving toward lateral movement, command and control, exfiltration, and impact.

The updated result shows that the stage-aware designs stop more lifecycles at Discovery than the No Stage setting. The Discovery stopping share is 56.5% for Technique Progress and 57.9% for Tactic, compared with 45.1% for No Stage. This indicates that stage information helps the agent take more suitable actions before the attack continues into later tactics.

However, No Stage has a higher stopping share at Lateral Movement / Collection, reaching 19.0%, compared with 5.2% for Technique Progress and 6.8% for Tactic. This does not necessarily mean No Stage is better. Instead, it suggests that without stage-aware guidance, more attacks continue beyond Discovery and are only stopped at a later tactic. In contrast, the stage-aware settings shift more stopping behavior earlier to Discovery, which is more desirable for reducing attack progression.

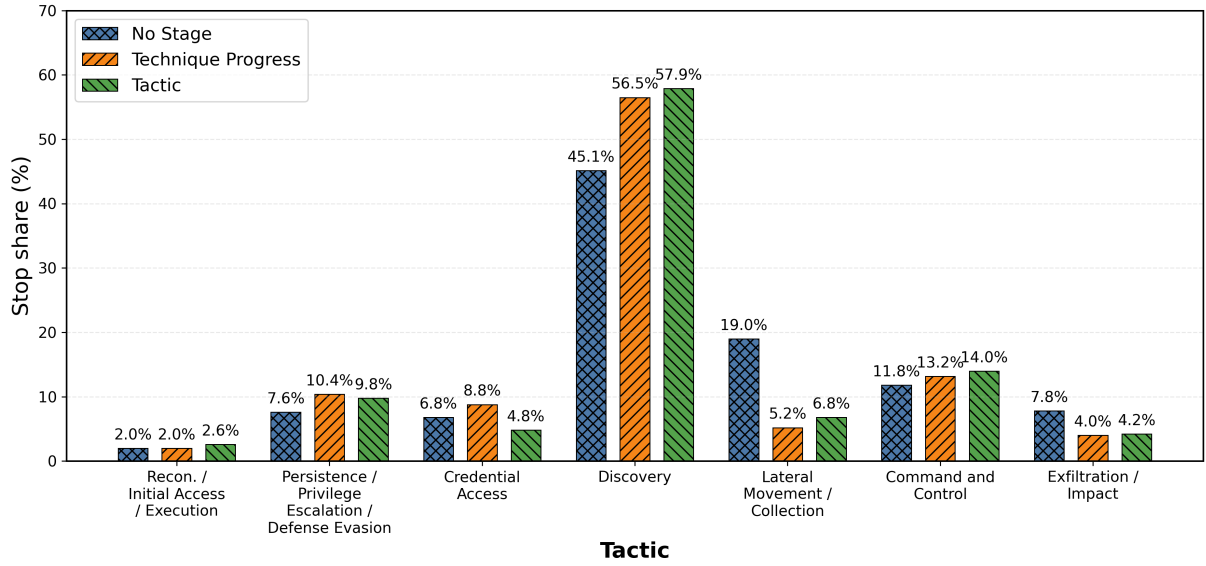


Figure 6.7: Lifecycle stopping distribution by tactic under different RL stage designs.

6.5 Response Model Comparison: RL vs. LLM

The third result compares the trained RL response policy with prompt-based LLM response models. The LLM baselines receive response prompts that include the same major response context used by the RL agent, including the current stage, system artifacts, system status, and available response actions. This comparison evaluates whether prompt-based action selection can match a policy trained through repeated environment feedback.

6.5.1 Recovery Success Rate

Figure 6.8 compares the recovery success rate of RL and LLM response models. The RL response agent reaches a recovery success rate of about 97.5%, while Gemini reaches 88.9% and OpenAI reaches 60.1%. This shows that the trained RL policy restores compromised hosts more reliably than the prompt-based LLM baselines.

The difference comes from how the models learn the response behavior. The LLM can read the response context and produce a plausible action, but it does not learn directly from whether the action actually restores the system in the environment. In contrast, the RL agent receives reward feedback after each action and updates its policy based on recovery outcomes. Therefore,

the RL policy becomes more consistent with the artifact-clearing logic used in the response environment.

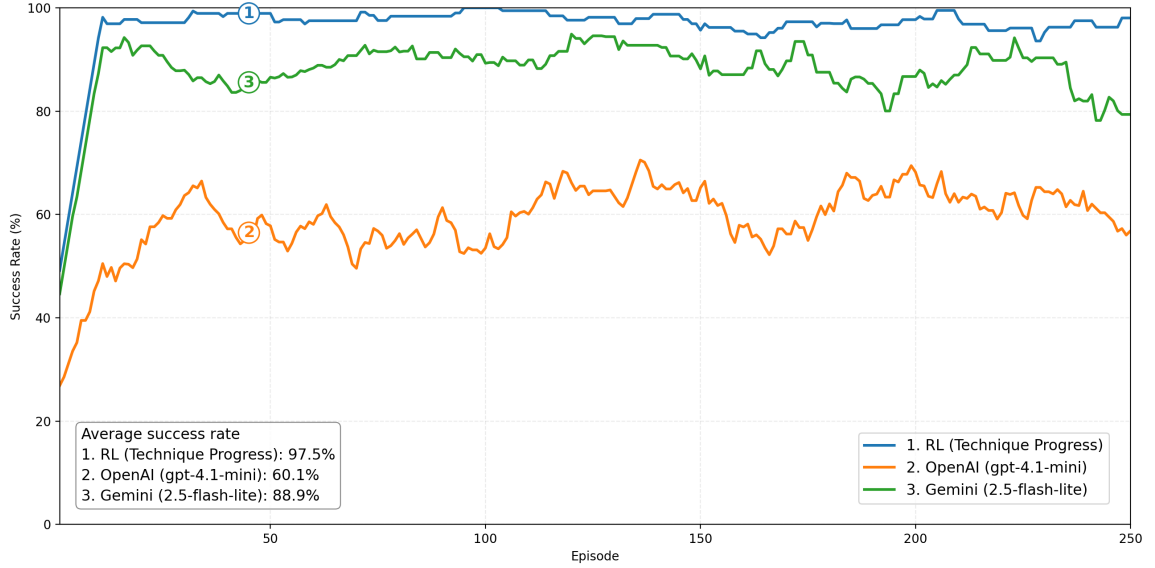


Figure 6.8: System recovery success rate of RL and LLM response models.

6.5.2 Stage-Aware Action Behavior

Figure 6.9 compares the action-impact distribution of the RL response policy and the LLM response baselines. In the Early stage, all models keep high-impact actions rare, which means that they generally avoid aggressive containment at the beginning of the lifecycle. RL shows the most conservative early-stage behavior, selecting 94.8% low-impact actions, 5.2% medium-impact actions, and only 0.1% high-impact actions. Gemini is also mostly conservative, with 90.1% low-impact actions. OpenAI, however, selects fewer low-impact actions at 68.8% and uses medium-impact actions more often at 30.1%. This suggests that the prompt-based model can follow the low-disruption principle, but its early-stage action strength is less consistent than the trained RL policy.

In the Mid stage, the models begin to show different response tendencies. RL selects 82.2% low-impact actions and 17.2% medium-impact actions, while keeping high-impact actions very low at 0.7%. Gemini shows a similar low-impact tendency with 82.7% low-impact actions, but it also selects 7.4% high-impact actions. OpenAI is more aggressive in the Mid stage, selecting 64.9% low-impact actions, 27.6% medium-impact actions, and 7.5% high-impact actions. This

indicates that the LLM baselines can vary in action strength across models, but the variation is not directly optimized by environment feedback.

The clearest difference appears in the Late stage. RL reduces its low-impact share to 53.5% and increases stronger mitigation, selecting 16.4% medium-impact actions and 30.0% high-impact actions. In contrast, the LLM baselines remain mostly low impact even in the Late stage: OpenAI selects 83.4% low-impact actions and only 4.3% high-impact actions, while Gemini selects 89.6% low-impact actions and only 4.7% high-impact actions. This shows that RL adapts more clearly to lifecycle severity, whereas the LLM response models remain conservative even when the attack reaches a higher-risk stage.

Overall, the result highlights the advantage of reward-based response learning. The RL agent learns a stage-aware policy that stays conservative in the Early stage but escalates response strength in the Late stage when stronger containment becomes more acceptable. The LLM baselines can generate plausible actions from the prompt, but prompt instructions alone do not provide the same policy optimization as repeated reward feedback from the response environment.

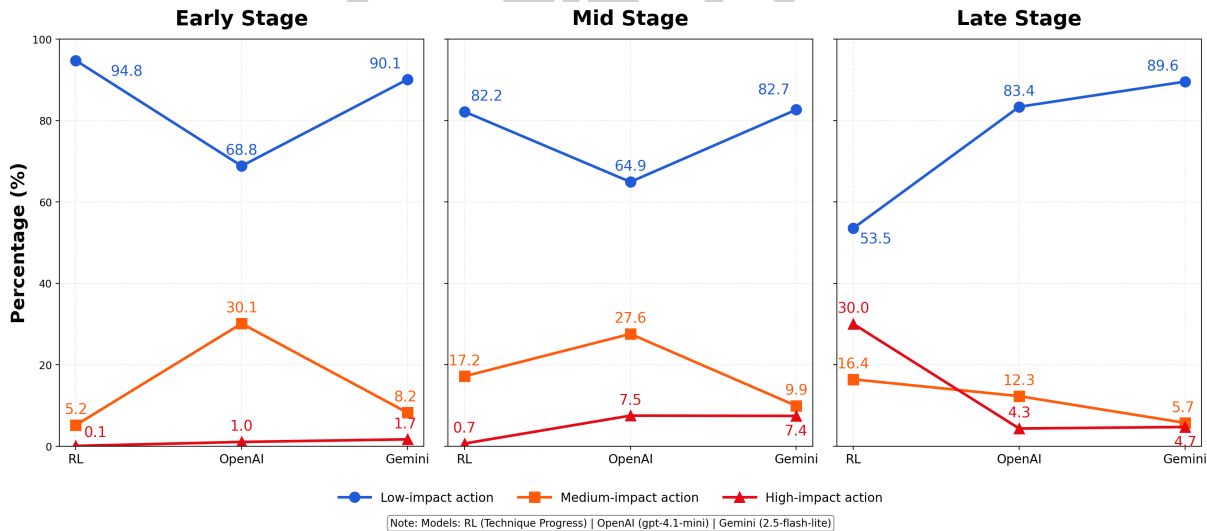


Figure 6.9: Action-impact distribution of RL and LLM response models across attack stages.

6.5.3 Lifecycle Stopping Comparison

Figure 6.10 compares the lifecycle stopping distribution of RL and LLM response models. The main difference appears at Discovery. RL stops 58.1% of lifecycles at Discovery, while OpenAI stops 24.2% and Gemini stops 30.7%. This shows that the trained RL policy stops more attacks before they progress into later and higher-risk tactics.

The LLM baselines stop more lifecycles at a later tactic, especially Command and Control. OpenAI stops 41.2% of lifecycles at Command and Control, while Gemini stops 37.2%. In comparison, RL stops 14.8% at this tactic. This means that LLM-based response often delays effective mitigation until the attack has already reached a more dangerous stage.

This difference comes from the learning mechanism. The LLM baselines can follow the prompt and choose reasonable actions, but they do not improve through environment reward feedback. The RL agent repeatedly observes whether actions restore the system and stop future techniques, so it becomes more consistent at stopping the lifecycle earlier.

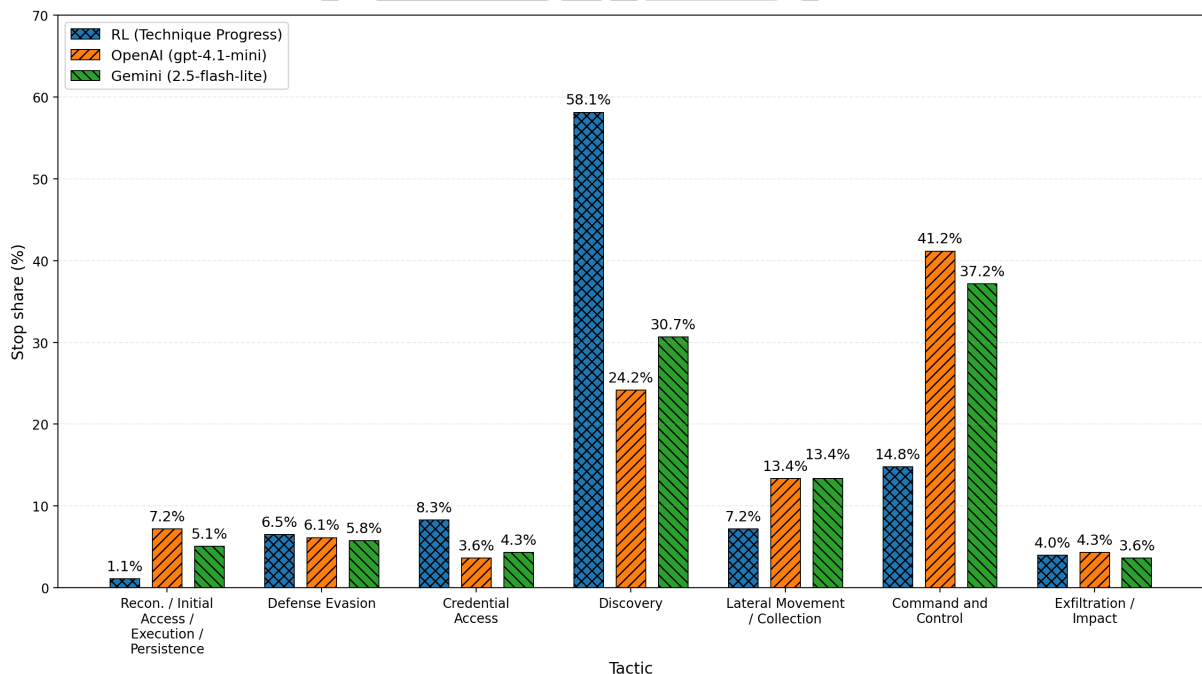


Figure 6.10: Lifecycle stopping distribution by tactic for RL and LLM response models.

6.6 End-to-End Architecture Comparison

The final result compares the runtime efficiency of the complete detection–response pipeline. TRACE streamlines Transformer-based technique detection with RL-based response. The baseline architecture uses LLM-based detection and LLM-based response. The comparison focuses on detection latency, response latency, and total end-to-end latency.

Figure 6.11 shows that Transformer+RL has much lower total latency than the LLM-based detection–response pipelines. The Transformer+RL pipelines require only 13.97–16.95 ms in total, while the LLM-based pipelines require 1284.29–2041.29 ms. The fastest configuration is SecRoBERTa + RL, with 2.78 ms/sample detection latency, 11.2 ms/sample response latency, and 13.97 ms total latency.

The main bottleneck is the LLM response stage. LLM detection takes about 62.29–62.35 ms/sample, but LLM response requires about 1222–1980 ms/sample, which dominates the end-to-end runtime. In contrast, RL response requires only about 11.2 ms/sample. This large latency gap shows why an LLM-only detection–response pipeline is less suitable for real-time SOC operation, especially when alerts arrive continuously and response actions must be selected quickly.

This result supports the architectural design of TRACE. A specialized Transformer detector can provide fast technique classification, while the RL agent can select actions with low response latency. By separating detection and response into specialized models, TRACE achieves both strong decision quality and practical end-to-end speed.

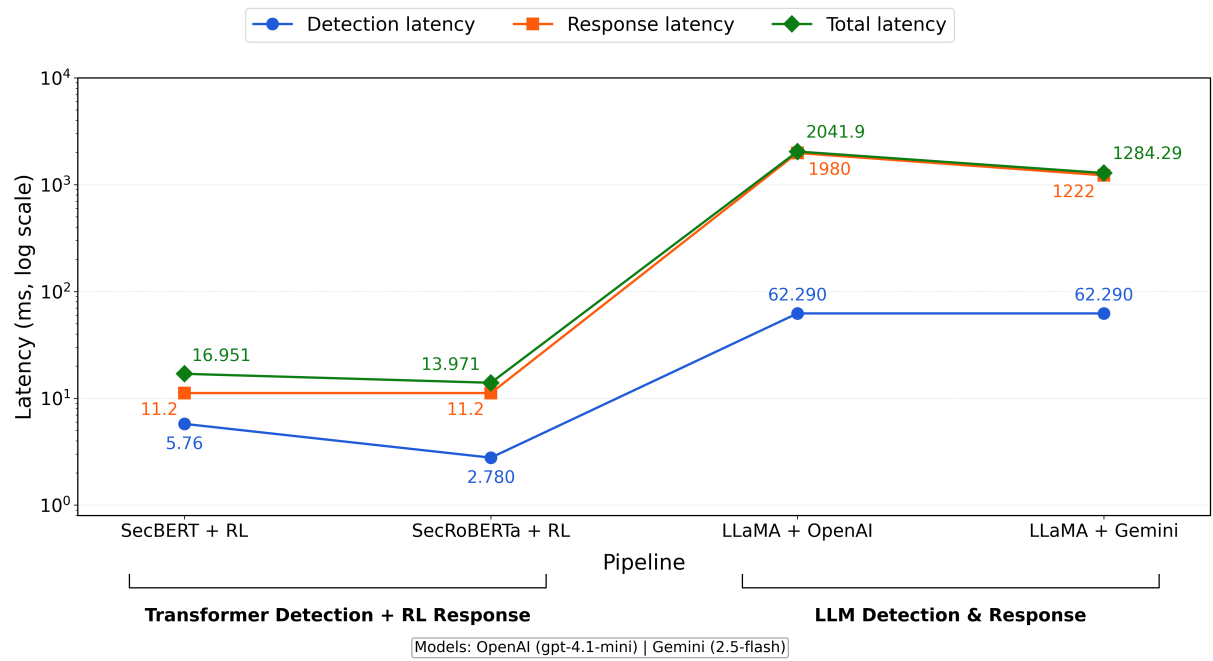
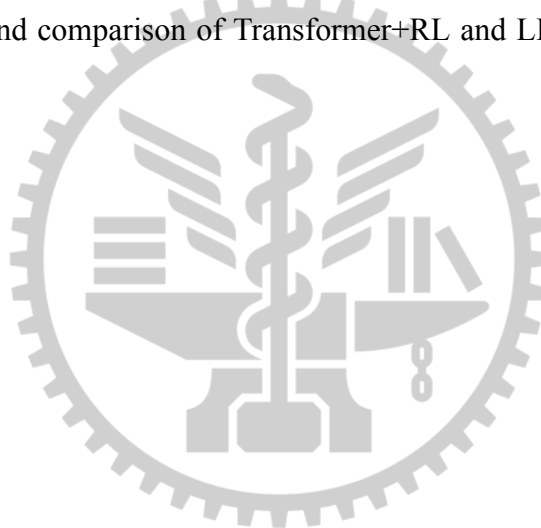


Figure 6.11: End-to-end comparison of Transformer+RL and LLM-based detection–response pipelines.



Chapter 7

Conclusion and Future Work

This thesis presented TRACE, a Transformer–RL Automated Cybersecurity Engine for automated threat mitigation in Security Operations Center (SOC) environments. TRACE streamlines two specialized modules: a Transformer-based detector for identifying the current MITRE ATT&CK technique from multi-source alerts, and a reinforcement learning-based response agent for selecting mitigation actions based on the attack stage, system artifacts, and system status. This design allows detection and response to work as one automated pipeline while keeping each module focused on its own task.

The detection results show that task-specific Transformer models provide a better balance between performance and latency than LLM-based detection. Transformer models achieve F1-scores of 0.856–0.866, while the LLM baselines achieve slightly lower F1-scores of 0.833–0.834. More importantly, Transformer inference requires only 2.78–5.76 ms per sample, compared with 62.29–62.35 ms for the LLM baselines. This shows that fine-tuned Transformer models are more suitable for real-time SOC detection, where a large number of alerts must be processed continuously.

The response results show that the DDQN response agent learns a stable and effective mitigation policy. The recovery success rate starts at approximately 40%–50% during early training, then becomes stable above 90% after sufficient learning. The final trained RL policy reaches a system recovery success rate of 97.5%. The stage-aware reward design also helps the agent adjust action strength according to attack progression. In the early stage, the agent mainly selects low-impact actions, reaching 91.9% for No Stage, 92.7% for Technique Progress, and 90.7% for Tactic. In the late stage, the stage-aware designs select stronger actions more often, with high-impact actions reaching 29.2% for Technique Progress and 28.0% for Tactic, compared with only 1.2% for No Stage. This indicates that stage-aware learning helps the response policy

remain conservative early while becoming more aggressive when the attack reaches later stages.

The lifecycle stopping results further show that stage-aware response improves attack suppression. Both Technique Progress and Tactic stop many lifecycles at Discovery, with 56.5% and 57.9%, respectively, compared with 45.1% for No Stage. In contrast, No Stage stops more lifecycles later at Lateral Movement / Collection, reaching 19.0%, compared with 5.2% for Technique Progress and 6.8% for Tactic. This suggests that stage-aware response helps stop attacks earlier, while the no-stage design more often allows the lifecycle to continue into later tactics before stopping.

Compared with prompt-based LLM response models, the RL response agent provides stronger recovery and more adaptive response behavior. RL reaches 97.5% recovery success, compared with 88.9% for Gemini and 60.1% for OpenAI. In late-stage attacks, RL escalates response strength more clearly, selecting 30.0% high-impact actions and 16.4% medium-impact actions. In contrast, OpenAI and Gemini remain mostly low impact in the Late stage, with 83.4% and 89.6% low-impact actions, and only 4.3% and 4.7% high-impact actions, respectively. For lifecycle stopping, RL stops more attacks at Discovery, reaching 58.1%, compared with 24.2% for OpenAI and 30.7% for Gemini. The LLM baselines stop more lifecycles later at Command and Control, reaching 41.2% for OpenAI and 37.2% for Gemini, while RL is only 14.8%. This shows that RL benefits from reward feedback and learns from environment outcomes, whereas prompt-based LLM response does not update its behavior through interaction.

The end-to-end architecture comparison shows that the TRACE Transformer–RL pipeline is much faster than the LLM-based detection–response pipelines. Transformer–RL achieves 13.97–16.95 ms total latency, while the LLM pipelines require 1284.29–2041.29 ms. The main bottleneck of the LLM pipeline comes from response inference, which requires 1222–1980 ms, compared with approximately 11.2 ms for RL response. Overall, the results show that TRACE provides accurate technique detection, reliable system recovery, stage-aware mitigation, earlier lifecycle stopping, and low-latency operation for real-time SOC threat mitigation.

Future Work. Several extensions can further improve the generalizability and deployment readiness of TRACE. First, future work can expand the attack lifecycle dataset with more CTI reports, additional MITRE ATT&CK techniques, more tactic combinations, and more diverse enterprise attack scenarios. With a broader dataset, TRACE can support online or continual learning, allowing the detection model and response agent to update incrementally as new lifecycles appear instead of requiring full retraining from scratch. Second, TRACE can be improved with an alert validation or false-positive filtering module before technique detection and response. Since real SOC environments often contain benign or low-confidence alerts, this module can first estimate whether an alert window is likely to be a true attack or a false positive, which can reduce unnecessary response actions, improve analyst trust, and make automated mitigation safer for practical deployment.



References

- [1] F. J. Aparicio-Navarro, K. G. Kyriakopoulos, I. Ghafir, S. Lambotharan, and J. A. Chambers, “Multi-stage attack detection using contextual information,” in *MILCOM 2018 - 2018 IEEE Military Communications Conference (MILCOM)*, Los Angeles, CA, USA, 2018, pp. 1–9.
- [2] H. Li, “Research on decision optimization of network security expert system based on multi-source heterogeneous data,” *Journal of Combinatorial Mathematics and Combinatorial Computing*, vol. 127, pp. 8683–8700, 2025.
- [3] S. Tariq, M. B. Chhetri, S. Nepal, and C. Paris, “Alert fatigue in security operations centres: Research challenges and opportunities,” *ACM Computing Surveys*, vol. 57, no. 9, pp. 1–38, 2025, article 224.
- [4] Y.-D. Lin, C.-Y. Chang, D. Sudyana, Y.-C. Lai, R.-H. Hwang, P.-C. Lin, E. H.-K. Wu, and W.-B. Lee, “LLM-driven alert correlation and lifecycle prediction with threat intelligence for attack forensics,” 2026, unpublished manuscript.
- [5] A. R. Muhammad, P. Sukarno, and A. A. Wardana, “Integrated security information and event management (SIEM) with intrusion detection system (IDS) for live analysis based on machine learning,” *Procedia Computer Science*, vol. 217, pp. 1406–1415, 2023.
- [6] K. Malialis and D. Kudenko, “Distributed response to network intrusions using multiagent reinforcement learning,” *Engineering Applications of Artificial Intelligence*, vol. 41, pp. 270–284, 2015.
- [7] K. Hughes, K. McLaughlin, and S. Sezer, “A model-free approach to intrusion response systems,” *Journal of Information Security and Applications*, vol. 66, p. 103150, 2022.
- [8] I. Ismail, R. Kurnia, Z. A. Brata, G. A. Nelistiani, S. Heo, H. Kim, and H. Kim, “Toward robust security orchestration and automated response in security operations centers with

a hyper-automation approach using agentic artificial intelligence,” *Information*, vol. 16, no. 5, p. 365, 2025.

- [9] IBM, “What is a security operations center (soc)?” Online. Available: <https://www.ibm.com/think/topics/security-operations-center>, 2025.
- [10] The MITRE Corporation, “Mitre att&ck®,” Online. Available: <https://attack.mitre.org/>, 2025.
- [11] Palo Alto Networks, “What is cyber threat intelligence (cti)?” Online. Available: <https://www.paloaltonetworks.com/cyberpedia/what-is-cyberthreat-intelligence-cti>, 2025.
- [12] Y. T. Huang, R. Vaitheeshwari, M. C. Chen, Y. D. Lin, R. H. Hwang, P. C. Lin, Y. C. Lai, E. H. K. Wu, C. H. Chen, Z. J. Liao, and C. K. Chen, “MITREtrieval: Retrieving MITRE techniques from unstructured threat reports by fusion of deep learning and ontology,” *IEEE Transactions on Network and Service Management*, vol. 21, no. 4, pp. 4871–4887, 2024.
- [13] IBM, “What is a transformer model?” Online. Available: <https://www.ibm.com/think/topics/transformer-model>, 2025.
- [14] IBM, “What are large language models?” Online. Available: <https://www.ibm.com/think/topics/large-language-models>, 2025.
- [15] IBM, “What is fine-tuning?” Online. Available: <https://www.ibm.com/think/topics/fine-tuning>, 2025.
- [16] IBM, “What is reinforcement learning?” Online. Available: <https://www.ibm.com/think/topics/reinforcement-learning>, 2025.
- [17] IBM, “What are ai agents?” Online. Available: <https://www.ibm.com/think/topics/ai-agents>, 2026.
- [18] J. Lee, J. Kim, J. Kim, and K. Han, “Cyber threat detection based on artificial neural networks using event profiles,” *IEEE Access*, vol. 7, pp. 165 607–165 626, 2019.

- [19] K. Hughes, K. McLaughlin, and S. Sezer, “Dynamic countermeasure knowledge for intrusion response systems,” in *2020 31st Irish Signals and Systems Conference (ISSC)*, Letterkenny, Ireland, 2020, pp. 1–6.
- [20] E. Iturbe, A. Rego, O. Llorente-Vazquez, E. Rios, C. Dalamagkas, D. Merkouris, and N. Toledo, “Reinforcement learning in action: Powering intelligent intrusion responses to advanced cyber threats in realistic scenarios,” *Expert Systems With Applications*, vol. 296, p. 129168, 2026.

